

Architettura

01_Architettura

Questo file è la rielaborazione delle slide [01_Architettura.pdf](#)

Questo file è la rielaborazione delle slide [01_Architettura.pdf](#)

Un'**algoritmo** è un insieme finito di istruzioni usate per la risoluzione di un certo lavoro. Quest'ultimo molte volte viene definito come una particolare funzione parziale di una [macchina di Turing](#) (Alan Turing, matematico britannico) oppure come un programma di una [macchina di Von Neumann](#) (matematico statunitense).

Le funzioni di base di un'elaboratore possono essere riassunte in 4 macro aree:

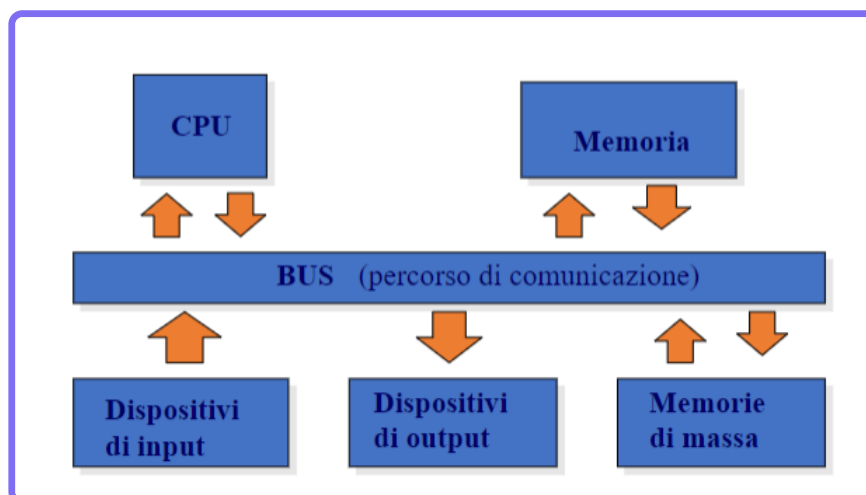
- memorizzazione dei dati
- elaborazione dei dati
- trasferimento dei dati
- controllo

Un'esempio di elaboratore è il **computer**, il computer è una macchina che computa ovvero che esegue un certo algoritmo (ovviamente scritto in modo che la macchina stessa lo possa interpretare). Esistono vari tipi di computer:

- Laptop
- Server
- Desktop
- Tablet/Smartphone

Un qualsiasi computer moderno segue l'architettura della macchina di Von Neumann

Macchina di Von Neumann



Questo tipo di architettura viene chiamata Macchina di Von Neuman (questa è l'architettura alla quale fa riferimento la definizione di algoritmo sopracitata). Di seguito una descrizione di tutti i componenti:

- La **CPU, o Central Processing Unit**, è l'unità centrale di elaborazione di un computer. È il componente principale che esegue le istruzioni dei programmi, gestisce le operazioni logiche e aritmetiche, e coordina il funzionamento delle altre parti del sistema. Come memoria di lavoro usa i registri e la cache. La sua velocità si misura in numero di cicli al secondo (MHz o GHz). La CPU è formata da 4 parti fondamentali:
 - il **program counter**: una locazione di memoria che contiene l'indirizzo dell'istruzione da eseguire
 - il **registro delle istruzioni**: una locazione di memoria contenente l'istruzione da eseguire
 - **ALU (Arithmetic logic unit)**: un sistema che esegue le operazioni aritmetiche e logiche
 - **CU (Control Unit)**: Il sistema di controllo gestisce il flusso di esecuzione delle istruzioni. Fa sì che ogni parte del processore faccia il suo lavoro nel momento giusto, attraverso una serie di cambiamenti di stato.
- La **memoria**, è un deposito di dati e di istruzioni da eseguire, ne esistono di 3 tipi:
 - **ROM, o Read Only Memory**, è una memoria di sola lettura non volatile dove si trovano tutte varie informazioni come le istruzioni usate per l'avvio del pc, o dei parametri necessari per il corretto funzionamento del dispositivo
 - **RAM, o Random Access Memory**, è una memoria volatile utilizzata per immagazzinare tutte quelle istruzioni che poi vengono eseguite dal processore
 - La **cache** è una memoria ad alta velocità che si trova all'interno o vicino alla CPU e viene utilizzata per immagazzinare temporaneamente i dati e le istruzioni più frequentemente utilizzate. La sua funzione principale è quella di ridurre i tempi di accesso alla RAM migliorando così le prestazioni del sistema e quindi riducendo il **Bottleneck** tra CPU e RAM. [Approfondimento qui](#)
- **Dispositivi di input**: come una tastiera ed un mouse []]
- **Dispositivi di output**: come un monitor o una stampante
- **Memoria di massa**: un tipo di memoria non volatile che viene usata per immagazzinare grandi file, e ovviamente molto più lenta di qualsiasi altro tipo di memoria sopracitata. La memoria di massa è fondamentale per garantire l'archiviazione stabile di software, documenti, immagini e altri dati digitali, rendendoli accessibili nel tempo. Degli esempi sono:
 - **Hard disk drive (HDD)**: basato su dischi magnetici rotanti.
 - **Solid State Drive (SSD)**: usa chip di memoria flash, più veloce e resistente rispetto agli HDD.
- **BUS**, tutte queste componenti comunicano tra di loro attraverso i bus. I principali tipi di bus vengono usati per inviare dati o segnali di controllo. [Approfondimento qui](#)

La macchina di von Neumann viene definita logicamente come una terna, ovvero un'insieme di 3 elementi:

- **N** = {0,1,2,3} ovvero l'insieme dei numeri naturali (l'alfabeto della macchina)

- **IS** = {ZERO, INC, SOM, SOT, MOL, DIV, UGUALE, MINORE, SALCOND, ALT} è l'Instruction Set ovvero un'insieme di istruzioni che la macchina può usare
- **P** = {I0, I1, I2, I3, ..., I|P| - 1} è una sequenza finita di istruzioni prese dall'insieme IS, questo insieme si chiama programma

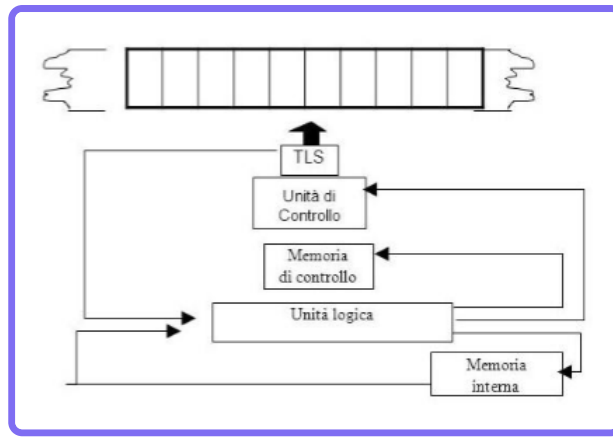
Come possiamo ben notare un programma eseguibile dalla macchina von Neumann consiste in una lista di istruzioni che devono essere eseguite dal processore. Ogni istruzione viene sottoposta al ciclo macchina ovvero una serie di passaggi impiegati per l'esecuzione dell'istruzione stessa, di seguito i passaggi appena citati:

1. **Legge il contenuto del program counter**: ovvero l'indirizzo della prossima istruzione da eseguire.
2. **Caricamento nel registro delle istruzioni (fetch)**: Il processore va a recuperare l'istruzione dalla memoria, utilizzando l'indirizzo letto da program counter. Questa istruzione viene quindi inserita nel **registro delle istruzioni**.
3. **Decodifica dell'istruzione**: Una volta che l'istruzione è stata caricata, il processore la decodifica, per capire di che tipo di istruzione si tratta
4. **Invio all'ALU**: Se l'istruzione richiede un'operazione aritmetica o logica, la **unità logico-aritmetica** (ALU) riceve l'istruzione e i dati necessari per eseguire l'operazione.
5. **Accesso ai dati**: Se l'istruzione da eseguire richiede dei dati la control unit li recupera dalla memoria. Se l'istruzione è del tipo SOM(M1, M2) ad esempio, significa che M1 e M2 sono indirizzi in memoria da cui vengono presi i dati.
6. **Esecuzione**: L'ALU effettua l'operazione richiesta (ad esempio, somma o confronto) utilizzando i dati forniti.
7. **Memorizzazione del risultato**: Una volta ottenuto il risultato, viene registrato nella **locazione di memoria** specificata dall'istruzione.
8. **Aggiornamento del contatore**: Il program counter viene incrementato per puntare all'istruzione successiva.
9. **Ripetizione del ciclo**: Questo ciclo continua fino a quando non viene incontrata un'istruzione speciale che ferma o altera l'esecuzione, come un'istruzione ALT o un salto condizionato che modifica il flusso del programma.

Un altro modello di elaboratore dalla quale Von Neumann prese spunto fu la macchina di Turing

Macchina di Turing

Una **macchina di Turing (o MdT)** è stata inventata da Alan Turing nel 1936. Questo modello è fondamentale nella teoria della computabilità e fornisce una rappresentazione astratta di come funzionano i calcolatori. Formalmente viene definita in questo modo:



Di seguito una descrizione di tutti i componenti:

- **Nastro:** Un'unità di memoria esterna infinita, suddivisa in celle, ogni cella contiene un simbolo oppure è vuota.
- **Testina di lettura/scrittura (TLS):** Un dispositivo che interagisce direttamente con il nastro.
- **Unità di memoria interna:** Una struttura che memorizza lo stato interno della macchina.
- **Unità di calcolo:** Un componente che esegue le operazioni di base.
- **Unità di controllo:** Il "cervello" della macchina, che coordina le altre unità.
- **Unità di logica:** Un componente che si occupa delle operazioni logiche.

Il comportamento di una MdT può essere programmata definendo un'insieme di regole, o quintuple di questo tipo:

- (stato-interno-corrente, simbolo-letto, prossimo-stato-interno, simbolo-scritto, direzione)

di seguito degli esempi:

- (0, A, 1, B, -) se la macchina si trova nello stato 0 e legge il simbolo A passa allo stato 1 e scrive sul nastro B e sta ferma
- (1, B, 0, A, >) se si trova nello stato 1 e legge il simbolo B passa allo stato 0 e scrive sul nastro A e si muove di una posizione a destra

È importante sottolineare come l'attenzione di Turing sia rivolta al processo di calcolo, **indipendentemente da come esso avviene fisicamente**. Una M.d.T è un dispositivo ideale, cioè indipendente da ogni sua possibile realizzazione fisica. Una funzione si dice **Turing-computabile** se almeno una MdT è in grado di computarla con un numero finito di passi.

Condizioni di finitezza

Una MdT per essere tale deve rispettare le condizioni di finitezza che sono:

- il numero di simboli che usa deve essere fissato e finito
- il numero di caselle del nastro osservabili in una volta è finito
- è possibile ricordare solo un numero finito di stadi precedenti
- le operazioni che può compiere sono:

1. Cambiare il contenuto di alcune caselle osservate
 2. Cambiare le caselle osservate
 3. Cambiare il proprio stato
 4. Osservare nuove caselle che si trovano ad una distanza prefissata dalla casella osservata
-

Macchina di Turing universale

Se supponiamo di avere una macchina di Turing senza limiti di spazio, di tempo e che non possa commettere errori quest'ultima sarà in grado di calcolare tutte le funzioni calcolabili in ogni singola macchina di Turing, questa macchina la chiamiamo **Macchina di Turing Universale (MdTU)**, inoltre deve rispettare sia la condizione di finitezza sopracitata ma anche la condizione di determinatezza spiegata di seguito

Condizione di determinatezza

Le azioni di una MdTU devono dipendere solo dai simboli contenuti nella casella osservata in quell'istante e dallo "Stato mentale" corrente, cioè da quello che ricorda dei calcoli precedenti

Da tutto questo Turing formula la seguente Tesi: *"ogni funzione parziale calcolabile con un algoritmo è una funzione parziale calcolabile da una macchina di Turing."* (appunto per questo all'inizio si parla dell'algoritmo come una funzione parziale della macchina di Turing). Questa tesi ci dice che **ogni Funzione parziale calcolabile da un algoritmo può essere calcolata da una macchina di Turing**, poiché ogni algoritmo può essere descritto in termini di un insieme finito di regole.

Bottleneck

La CPU è progettata per operare a velocità estremamente elevate, elaborando milioni di istruzioni al secondo. Tuttavia, la RAM, sebbene veloce, ha tempi di accesso più lunghi quando la CPU richiede dati o istruzioni deve attendere che questi vengano recuperati dalla RAM. Questo processo di attesa genera un rallentamento, poiché la CPU rimane inattiva in attesa dei dati necessari per continuare l'elaborazione. Per mitigare questo problema, le CPU fanno uso della cache, una memoria più veloce e più vicina al processore, dove vengono conservati i dati e le istruzioni più frequentemente utilizzati. Tuttavia, anche con l'uso della cache, il bottleneck rimane una preoccupazione, specialmente in scenari di carico elevato o quando vengono eseguiti più processi contemporaneamente. In questi casi, se la RAM non è in grado di tenere il passo con le richieste della CPU, si crea un rallentamento significativo, influenzando negativamente l'efficienza complessiva del sistema.

Il test di Turing

Turing cercò di rispondere alla domanda "can machines think?", per fare ciò formulò quello che viene chiamato test di Turing ovvero un giudice umano comunica con due interlocutori nascosti: uno è un essere umano, e l'altro è una macchina. Se il giudice non riesce a distinguere chi è la macchina e chi è l'umano basandosi solo sulle risposte fornite, allora si dice che la macchina ha **superato il test** e può essere considerata "intelligente". Fino a qualche mese fa nessuno era riuscito a superarlo tranne ChatGPT-4 di recente

Funzione parziale

Una funzione parziale è una funzione che non è necessariamente definita per ogni possibile input. Questo significa che per alcuni input, la funzione potrebbe non restituire alcun output. Quest'ultima vengono utilizzate nella definizione di una macchina di Turing universale perché riflettono la realtà infatti non tutte le computazioni terminano con successo. Questo rende il modello più realistico e potente perché è in grado di rappresentare qualsiasi tipo di computazione, sia quelle che vanno bene che quelle che vanno male

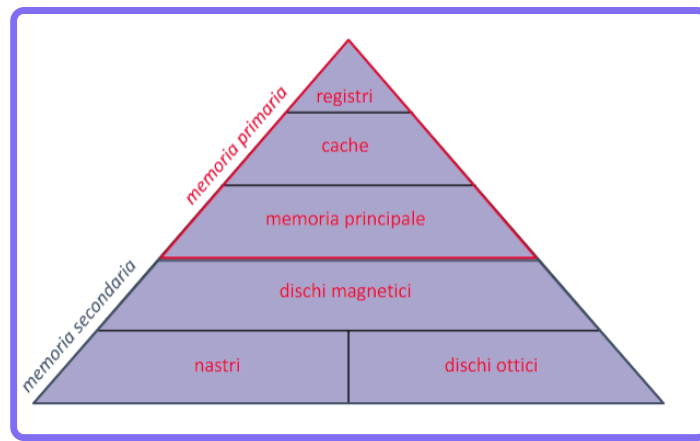
Processi sincroni/asincroni

- **Processi sincroni:** sono processi che hanno bisogno di essere "sincronizzati" perché sono propedeutici tra di loro.
- **Processi asincroni:** sono processi che non hanno bisogno di essere sincronizzati perché "lavorano da soli". La maggior parte dei processi asincroni vengono eseguiti dalle GPU quest'ultime hanno moltissimi core rispetto alle CPU proprio perché devono svolgere solo processi asincroni e quindi parallelizzabili, un'esempio dove questa proprietà è fondamentale è nella renderizzazione dello schermo, quest'ultimo è formato da matrici (la risoluzione ci indica quanto sono grandi Es: 720x480) dalla quale è possibile creare delle sottomatrici che vengono lavorate da processi indipendenti (e quindi asincroni), tutte queste sottomatrici vengono riunite nella matrice finale che sarà l'immagine che visualizzeremo a schermo (lo schermo a colori è formato da 3 matrici Red, Green, Blue che definiscono lo standard RGB).

02_Memoria

Questo file è la rielaborazione delle slide [02_Memoria.pdf](#)

La memoria dentro i nostri elaboratori ha una struttura piramidale, più si va in alto in questa piramide più la memoria diventa veloce ma si riduce di dimensione:



La **memoria principale** di un computer è costituita da un insieme di celle ordinate, ciascuna con una capacità di un byte e dotata di un indirizzo univoco. All'interno di queste celle vengono memorizzati sia le istruzioni dei software che i dati di input e output. Tuttavia, i contenuti delle celle vengono mantenuti solo finché la memoria è alimentata elettricamente, motivo per cui questa memoria viene definita **volatile**.

La maggior parte della memoria principale è realizzata con tecnologia **RAM** (*Random Access Memory*), e in particolare con una sua variante chiamata **DRAM** (*Dynamic RAM*). La DRAM è così denominata perché i dati in essa contenuti devono essere continuamente aggiornati. I tempi di lettura e scrittura della RAM, tuttavia, sono più lenti rispetto alle altre operazioni svolte da un elaboratore, creando quello che viene definito un **collo di bottiglia** (bottleneck).

Per mitigare questo problema, sono state sviluppate le **SRAM** (*Static RAM*), un tipo di RAM più veloce e stabile, ma anche più costoso. Per questo motivo, la SRAM viene utilizzata principalmente come memoria di transito tra la memoria principale e il processore, con il nome di **cache**.

Le memorie DRAM sono installate sulla scheda madre all'interno di slot chiamati **DIMM** (*Dual Inline Memory Module*), caratterizzati dalla presenza di circuiti su entrambe le facce del modulo.

Oltre alla RAM, la memoria principale include anche la **ROM** (*Read Only Memory*), una memoria di sola lettura utilizzata per conservare il **BIOS** (*Basic Input/Output System*). Il BIOS è fondamentale nella fase di avvio del computer, nota come **bootstrap**, durante la quale il dispositivo viene inizializzato correttamente. Successivamente, il BIOS esegue la fase di **POST** (*Power On Self Test*), che verifica il corretto funzionamento delle componenti hardware del sistema.

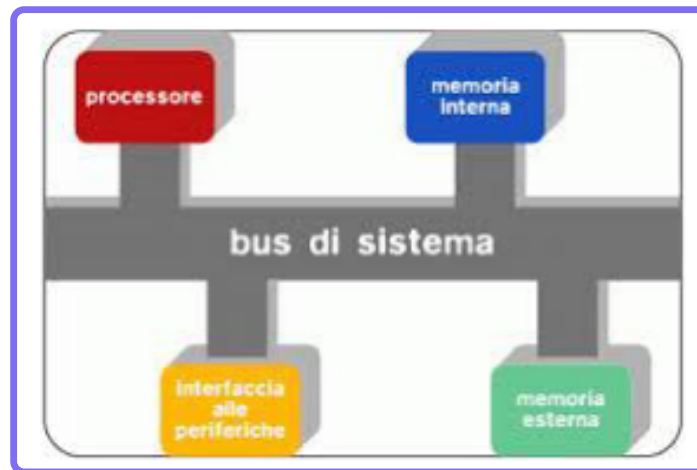
Anche se la ROM è progettata per essere di sola lettura, può essere riprogrammata in determinate circostanze, come durante le procedure di configurazione o aggiornamento del BIOS.

03_Bus

Questo file è la rielaborazione delle slide [03_Bus.pdf](#)

Il **bus** è l'unità di interconnessione che permette la comunicazione tra i vari componenti della macchina di Von Neumann. Si presenta come un fascio ordinato di linee, ciascuna delle quali può rappresentare un bit. Il bus funziona quindi come un mezzo di trasporto per le informazioni tra la CPU (definita **Master**), la memoria e i dispositivi di I/O (detti **Slave**).

Tipologie di collegamenti



Dalla figura possiamo individuare diversi tipi di collegamenti che si instaurano nel sistema:

- **Dal processore alla memoria**
- **Dalla memoria al processore**
- **Dal processore ai dispositivi di I/O**
- **Dai dispositivi di I/O al processore**

Quando il processore agisce come Master:

- Un'operazione che trasferisce un dato dal processore alla memoria si chiama **Write**.
- Un'operazione che trasferisce un dato dalla memoria al processore si chiama **Read**.

Il tempo che intercorre tra il segnale di invio dell'operazione e la sua effettiva esecuzione è chiamato **latenza**.

Suddivisione del bus

Per gestire correttamente il trasferimento dei dati, il bus è suddiviso in tre diverse tipologie di linee:

1. **Address Bus (ABus)**: trasporta le informazioni sugli indirizzi da utilizzare per il trasferimento.
2. **Data Bus (DBus)**: viene utilizzato per trasferire i dati.
3. **Control Bus (CBus)**: comprende linee dedicate a controllare il tipo e la direzione del trasferimento.

Linee del Control Bus

Le linee del **Control Bus** hanno compiti specifici:

- **I/O-Mem**: indica la direzione del trasferimento:
 - **Dal processore alle periferiche I/O**: valore impostato a **1**.
 - **Dal processore alla memoria**: valore impostato a **0**.
 - **R/W**: specifica il tipo di trasferimento:
 - **Read**: valore impostato a **1**.
 - **Write**: valore impostato a **0**.
 - **WAIT**: segnala lo stato del trasferimento:
 - **Trasferimento completato**: valore impostato a **1**.
 - **Trasferimento in corso**: valore impostato a **0**.
-

Sincronizzazione e velocità

La velocità del bus è regolata da un “orologio” interno, chiamato **clock**, che scandisce il tempo in modo costante, sincronizzando tutte le operazioni sul bus. Le dimensioni dei bus variano e influiscono sulle prestazioni del sistema, contribuendo al **collo di bottiglia** (bottleneck):

- **Address Bus (ABus)**: determina la quantità di memoria indirizzabile dai programmi. La memoria raggiungibile si calcola elevando 2 al numero di linee dell'ABus (es. un ABus a 32 bit permette di indirizzare 2^{32} byte di memoria).
 - **Data Bus (DBus)**: rappresenta il grado di parallelismo del processore, ovvero la quantità di dati che può elaborare simultaneamente.
Nota Bene: ogni singolo bus trasporta un solo bit, ma la combinazione di più linee consente il trasferimento parallelo di più bit.
-

Chipset e bus principali

Sulla scheda madre i bus di sistema sono difficili da identificare visivamente; il loro funzionamento è gestito da linee fisiche e chip dedicati, noti con il nome di **Chipset**. Tra i vari tipi di bus, il più importante è il **PCI** (*Peripheral Component Interconnect*), ormai largamente utilizzato. La sua variante più recente, **PCI Express**, è progettata per gestire elevate velocità di trasferimento ed è particolarmente diffusa per l'uso con schede video.

04_InputOutput

Questo file è la rielaborazione delle slide [04_InputOutput.pdf](#)

I dispositivi di input/output sono essenziali per acquisire dati dall'utente e per rappresentarli. Ogni dispositivo è associato a un proprio intervallo di indirizzi e utilizza linee di

sincronizzazione, chiamate **interrupt**, per comunicare con il processore. Esistono due principali tipi di interrupt:

- **INTR**: segnala una richiesta di interruzione al processore.
- **INTA**: conferma l'avvenuta gestione dell'interruzione.

Quando il dispositivo invia un segnale **INTR**, richiede al processore di sospendere temporaneamente la sua esecuzione per gestire l'interrupt, eseguendo una specifica procedura chiamata **Interrupt Service Routine** (ISR). Esistono anche modalità che permettono agli interrupt di bypassare la CPU e accedere direttamente alla memoria, tramite una procedura chiamata **Direct Memory Access** (DMA). La circuiteria dedicata a collegare il dispositivo al bus e a sincronizzarlo con le altre periferiche è detta **controller della periferica**. Un tempo, questi dispositivi erano fisicamente visibili all'esterno del computer, mentre oggi si trovano integrati nei circuiti interni. In passato, indirizzi, canali e linee dovevano essere configurati manualmente dall'utente o da un tecnico. Tuttavia, grazie alla tecnologia **Plug&Play** il BIOS, il sistema operativo e il firmware cooperano per gestire automaticamente la configurazione delle periferiche, sia all'avvio del computer che durante l'installazione di nuovi dispositivi.

Tra gli standard di connessione più diffusi, troviamo: **USB** (Universal Serial Bus), **FireWire** (o IEEE 1394), e **Ethernet** (802.x).

05_Processore

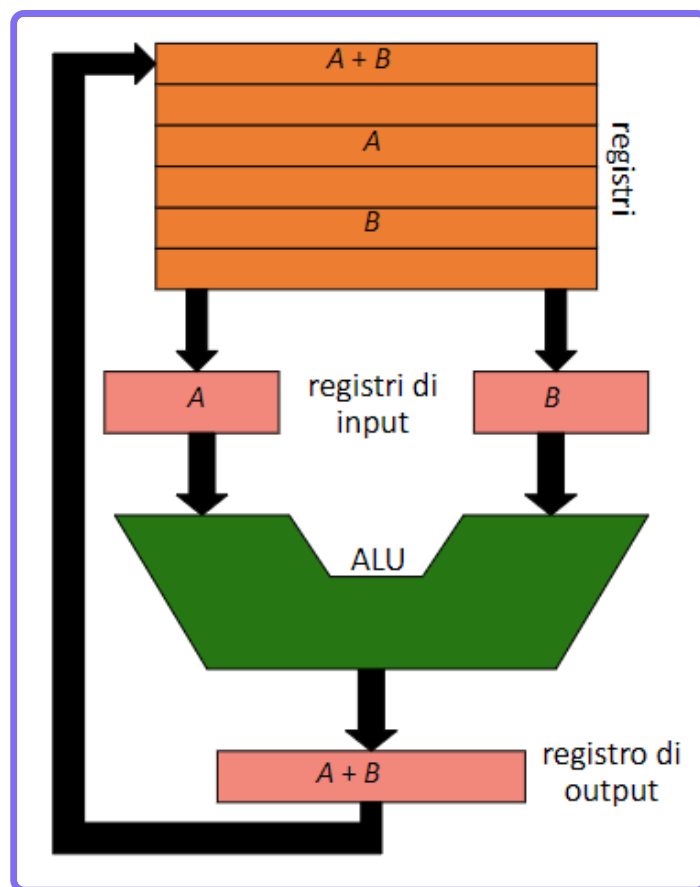
Questo file è la rielaborazione delle slide [05_Processore.pdf](#), approfondimento con codice assembly qui: [08_Struttura-base-del-processore](#)

Un processore è un singolo circuito integrato in grado di effettuare operazioni decisionali, di solito viene indicato con la sigla CPU (Central Processing Unit), il processore viene concettualmente diviso in 3 unità funzionali:

- **UC** (Control Unit): si affaccia sul [bus](#) e lo arbitra impostando i valori nelle linee ABus, DBus e CBus
- **Registri**: sono delle memorie di lavoro dove vengono conservati i dati presi dall'UC sul bus per poi poterli far usare dalla ALU durante l'esecuzione
- **ALU** (Arithmetic logic unit): è l'unità di esecuzione effettiva del processore

Ogni processore viene progettato con un set di istruzioni specifico denominato ISA (**Instruction Set Architecture**) o IS (**Instruction Set**), ogni istruzione dell'ISA è contraddistinta da un numero specifico denominato **Operation Code(OP)** ad ogni OP si associa una breve descrizione in lingua naturale che ne ricorda la funzione.

L'ALU, i registri e molti bus costituiscono il **data path**, che si presenta in questo modo:



Le istruzioni che devono essere usate dalla ALU, vengono caricate nei registri di input (che solitamente sono 2), la ALU fornisce il suo risultato nel registro di output (che solitamente è 1), infine il risultato verrà memorizzato nei registri, quello appena descritto è il ciclo del data path, ovvero la sequenza di passaggi che il computer esegue per eseguire un'operazione tramite la ALU (corrisponde alla fase di execute del ciclo macchina)

In base al tipo di Instruction set vengono definiti vari tipi di processore

- **CISC (Complex Instruction Set Computer):** basati su molte istruzioni complicate
- **RISC (Reduced Instruction Set Computer):** basati su poche e semplici istruzioni
- **CRISC (Complex Reduced Instruction Set Code):** quello più usato attualmente, è un architettura ibrida tra CISC e RISC

[Nota bene](#)

Il *Program Counter* (PC) è un registro della CPU che contiene l'indirizzo della prossima istruzione da eseguire.

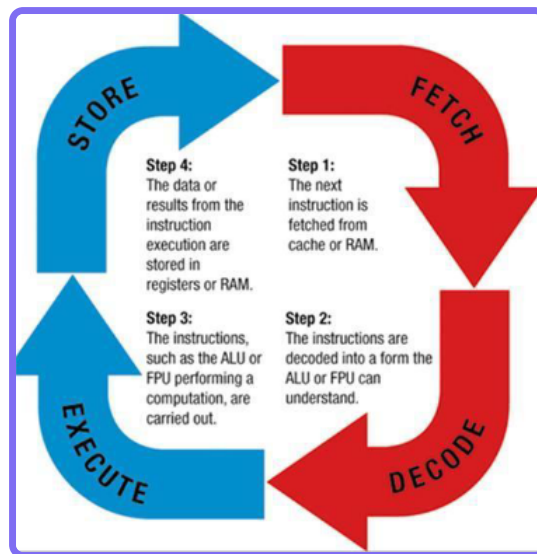
Un tipico processore opera seguendo una sequenza rigida di passi, che si ripetono fino all'arresto della macchina. Questi passi sono i seguenti:

- **Fetch:** L'Unità di Controllo (UC) carica nel bus degli indirizzi (ABus) il valore del Program Counter, attivando la lettura dalla memoria e caricando l'OpCode dell'istruzione corrente.
- **Decode:** L'UC, interpretando l'OpCode appena letto, determina la lunghezza dell'istruzione e quanti parametri sono necessari. Inizia quindi una fase intermedia,

chiamata Operand Fetch, per caricare gli operandi, che si trovano in indirizzi consecutivi a quello dell'OpCode. Gli operandi vengono quindi memorizzati nei registri.

- **Execute:** L'UC avvia il microprogramma corrispondente all'OpCode, utilizzando i parametri precedentemente caricati nei registri. La frequenza con cui vengono eseguiti questi microprogrammi è regolata dal clock della CPU, cioè la frequenza del microprocessore.
- **Store:** Al termine dell'operazione di Execute, i risultati, ora memorizzati nei registri, vengono trasferiti sul bus dall'UC. Questi possono essere scritti in memoria o inviati a dispositivi di input/output (PIO).

Questa sequenza continua a ripetersi per ogni istruzione fino al completamento o arresto del programma.



Questo ciclo può interrompersi se arriva alla CPU un segnale di INTR (della quale abbiamo parlato qui [04_InputOutput](#))

Ogni singola istruzione dell'IS di un processore è contraddistinta da un proprio Op. Code, una determinata lunghezza e un preciso numero di cicli di bus per il suo completamento, il tempo effettivo di esecuzione dell'istruzione è poco influente sulla durata totale di un'istruzione, infatti il bus è un'ordine di grandezza più lento rispetto alla CPU (questo va a creare il già discusso [collo di bottiglia](#))

CISC vs RISC

CISC

Le CPU CISC dispongono di un set di istruzioni ampio e complesso, progettato per eseguire operazioni di alto livello con un numero ridotto di istruzioni. Questo comporta però che ogni istruzione sia relativamente più lunga da eseguire, poiché richiede più cicli di clock e implica molteplici passaggi nel data path della CPU.

RISC

Le CPU RISC sono progettate con un set di istruzioni più semplice e costante, con comandi

che richiedono un solo ciclo di clock per essere eseguiti. Questo approccio rende il data path della CPU molto più lineare e veloce, facilitando una rapida esecuzione delle istruzioni.

Caratteristica	CISC	RISC
Pro	- Flessibilità: istruzioni complesse eseguono operazioni elaborate con poche istruzioni. - Compatibilità: architettura comune (es. x86), ampio supporto software. - Efficienza di memoria: istruzioni più complesse possono ridurre il numero complessivo di istruzioni necessarie.	- Alta velocità: istruzioni semplici eseguite in un singolo ciclo di clock, maggiore rapidità. - Efficienza energetica: consumo energetico ridotto per via della semplicità delle istruzioni. - Facilità di ottimizzazione: progettazione più semplice per prestazioni ottimali.
Contro	- Bassa efficienza: esecuzione più lenta per il numero elevato di cicli di clock necessari per ogni istruzione. - Maggiore consumo energetico: istruzioni complesse richiedono più energia. - Difficoltà di ottimizzazione: architettura complessa, meno ottimizzabile per velocità e prestazioni.	- Maggiore carico di programmazione: servono più istruzioni per compiti complessi. - Maggiore consumo di memoria: necessità di più istruzioni, che occupano più spazio. - Minor compatibilità software: adattamenti necessari per software sviluppato su CISC.

I processori moderni sono ibridi, ovvero usano un tipo di architettura chiamata **CRISC (Complex Reduced Instruction Set Computer)** cercando di ottenere i vantaggi di uno e dell'altro.

Come abbiamo capito il problema principale che causa un rallentamento della CPU è l'accesso alle risorse dal bus, per diminuire questi accessi e quindi aumentare il **throughput** si fa uso delle **cache**, una memoria "tampone" sulla quale vengono caricate le istruzioni più utilizzate dal processore, ad ogni operazione di lettura che fa il processore, controlla se l'informazione che gli serve è già nella cache in modo da poterci accedere in maniera più veloce, se non c'è il processore la richiede tramite il bus e sovrascrive l'istruzione meno usata che si trova nella cache con quella che ha appena richiesto. Esistono 3 tipi di cache:

- **L-1:** che si trova all'interno del processore
- **L-2:** che è collegata al processore
- **L-3:** sulla motherboard

Si può dire che la cache sia genericamente più veloce del bus, ma più si alza il livello di cache più questa differenza diminuisce.

Prefetch

Per aumentare il parallelismo di esecuzione viene usata la tecnica del prefetching che dice alla CPU che anziché limitarsi a caricare dalla memoria l'istruzione o il dato richiesto al momento, recuperi in anticipo anche le istruzioni o i dati immediatamente successivi. A

questo scopo, i processori dispongono di una "coda di prefetch", un buffer interno che può contenere una certa quantità di dati consecutivi (ad esempio, i 6 o 8 byte successivi a quello appena caricato). In questo modo, il processore effettua un singolo accesso alla memoria per caricare una serie di valori che possono essere utilizzati subito dopo, sia come istruzioni che come operandi.

Esistono 4 tipi di prefetch

1. **Prefetching hardware:** le CPU hanno un componente hardware al loro interno che rileva automaticamente i data pattern (dati che servono molte volte e seguono quindi una sorta di pattern nel loro utilizzo) nella memoria e li carica preventivamente nella cache.
 2. **Prefetching software:** nel prefetching software si usano delle variabili per suggerire alla CPU che certi dati saranno utilizzati molto spesso, quindi così la CPU li caricherà nella cache senza passare ogni volta dal BUS.
 3. **Tecnica di ottimizzazione in fase di compilazione :** è un metodo che in fase di compilazione identifica le linee di codice e i dati che vanno utilizzati di più. Questo è un modo per ottimizzare il codice. Funziona con compilatori moderni (C,C++ ecc...)
 4. **Prefetching con accesso alla linee di cache:** utilizza le linee di cache per l'accesso alla memoria, quindi trasporta dei blocchi fissi, da 64 byte solitamente, direttamente alla cache.
-

L'elaborazione di un'istruzione

L'elaborazione di un'istruzione da parte di un processore con pipeline si articola in cinque fasi principali:

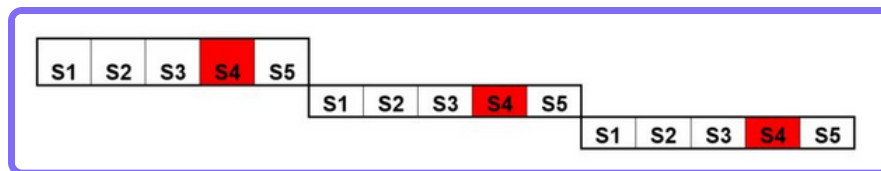
1. **IF (Instruction Fetch):** Recupero dell'istruzione dalla memoria.
2. **ID (Instruction Decode):** Decodifica dell'istruzione e lettura degli operandi dai registri.
3. **EX (Execution):** Esecuzione dell'istruzione.
4. **MEM (Memory Access):** Interazione con la memoria (valida solo per alcune istruzioni).
5. **WB (Write Back):** Scrittura del risultato nel registro.

Queste fasi, sequenziali e talvolta sovrapposte, permettono al processore di eseguire le operazioni previste dalle istruzioni.

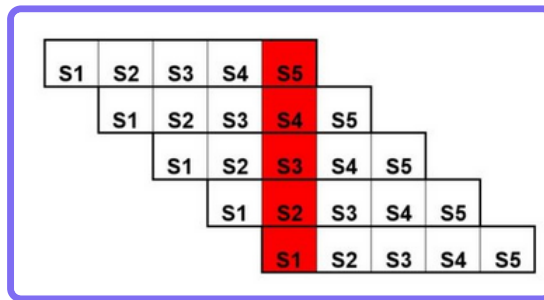
Pipeline

Alla coda di prefetch, è stato affiancato un sistema detto **pipeline** che ha lo scopo di sfruttare il concetto di catena di montaggio e quindi di parallelizzare più istruzioni possibili invece di eseguire un'istruzione completamente prima di eseguire la sua successiva. Basta che la prima istruzione sia in fase di decode che la seconda possa essere portata in fase di fetch.

- **Senza pipeline**



- Con pipeline



Come possiamo ben notare una pipeline a 5 stadi esegue 5 fasi contemporaneamente, questa tecnica sopprime al problema dell'accesso alla memoria precedentemente discusso.

L'uso della pipeline introduce anche delle problematiche:

1. **Data hazards:** si verifica quando un'istruzione richiede dei dati che vengono forniti da un'altra istruzione che non ha ancora finito (ritardi di elaborazione o blocchi)
2. **Control hazards:** si verifica quando la pipeline deve gestire dei salti condizionali (tipo il go-to)
3. **Structural hazards:** si verifica quando più istruzioni cercano di accedere alla stessa risorsa (problemi dei filosofi a cena)

Dopo aver dotato i primi processori di pipeline si ci è resi conto che la fase più lenta diventava la fase di execute (quella implementata nella ALU) per sopperire a questo problema si montano più ALU dentro ad un singolo processore. Un processore con più ALU è detto **Superscalare**

I problemi del prefetching e della pipeline

Tutte queste tecniche vengono vanificate da 2 situazioni:

- Istruzioni di salto: la pipeline viene del tutto persa se il salto ci porta ad un'istruzione lontana da quella che si trova attualmente in pipeline
- Dipendenza dei dati tra le istruzioni: la pipeline viene interrotta

Per cercare di arginare il problema delle istruzioni di salto sono stati introdotti nel processore dei moduli chiamati **Dynamic Branch Prediction** che si occupano di implementare l'**esecuzione predicativa**, una tecnica che fa uso di tabelle simili a memorie cache, per cercare di capire se un'istruzione di salto avverrà o meno (questa tecnica non è deterministica infatti si basa solo su una statistica storica), il problema principale di questa tecnica si ha quando la previsione fatta è sbagliata, infatti in quel caso la pipeline va ripristinata. L'esecuzione predicativa è anche nota come **esecuzione speculativa** quando va a fare una previsione sul codice che potrebbe non essere mai utilizzato.

Per cercare di arginare il problema delle dipendenze da dato si usa una tecnica chiamata **out of order execution** ovvero una tecnica che permette l'esecuzione delle istruzioni non necessariamente nell'ordine in cui sono state emesse, ma eseguendo le istruzioni prive di dipendenze mentre si attende il completamento di quelle con dipendenze, in questo modo la pipeline viene quasi del tutto conservata e non è necessario eseguire un flush con conseguente perdita di prestazioni. Ci sono però delle condizioni critiche:

- Le istruzioni "future" possono essere eseguite solo se non sono dipendenti a loro volta
- Quando il processore si trova fuori ordine e arriva un interrupt, il processore deve ripristinare il suo stato e riordinare anche il giusto flusso di esecuzione

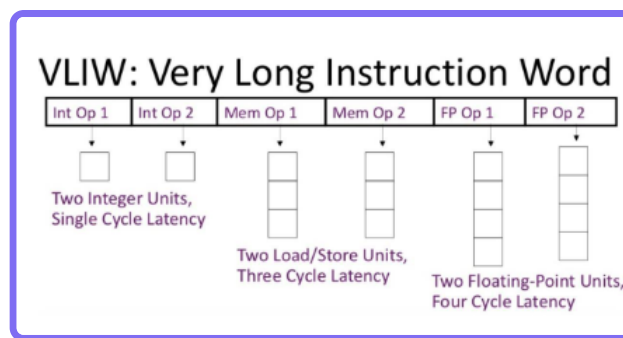
Per poter riordinare il giusto flusso di esecuzione i processori utilizzano una serie di registri d'appoggio (interni e invisibili al programmatore) su cui memorizzare i calcoli temporanei delle istruzioni fuori ordine. All'atto del riordinamento, per evitare di spostare i valori dai registri interni a quelli effettivamente usati nel data path, i processori sono in grado di rinominare i registri interni nei nomi dei registri effettivi, risparmiando il tempo del trasferimento questa tecnica si chiama **register renaming**.

Esistono 3 tipi di dipendenza del dato:

- **RAW (Read After Write):** Si verifica quando un'istruzione legge un dato che è stato scritto da una precedente istruzione. È chiamata anche dipendenza vera o *data dependency*, perché un dato deve essere prodotto prima di essere letto.
 - Esempio: Istruzione 1: $A = 5$, Istruzione 2: $B = A + 1$. L'istruzione 2 dipende dal valore di A prodotto dalla prima.
- **WAW (Write After Write):** Si verifica quando due istruzioni scrivono nello stesso dato. La seconda scrittura deve avvenire dopo la prima per non sovrascrivere accidentalmente il valore prodotto dalla prima. Questa è una dipendenza anti o di output.
 - Esempio: Istruzione 1: $A = 5$, Istruzione 2: $A = 10$. Qui la seconda scrittura sovrascrive il valore di A della prima istruzione.
- **WAR (Write After Read):** Si verifica quando una istruzione scrive su un dato che viene letto da una precedente istruzione. Questa dipendenza è detta anti-dipendenza, perché la scrittura deve avvenire solo dopo la lettura per evitare che il valore letto sia alterato.
 - Esempio: Istruzione 1: $B = A + 1$, Istruzione 2: $A = 3$. La seconda istruzione deve aspettare che la prima legga A prima di scrivere su di esso.

Very long Instruction Word

Anche se non esplicitamente, tutte queste innovazioni (pipeline, super-scalarità, predicazione, esecuzione fuori ordine) cercano di implementare un modello di esecuzione parallelo molto studiato nei centri di calcolo, e denominato VLIW (Very Long Instruction Word). In questo modello, oltre alla parallelizzazione dell'esecuzione ottenuta in hardware, si presuppone che lo stesso codice esecutivo generato dai compilatori sia «pre-cucinato» per essere parallelizzato ottimamente dalle CPU.

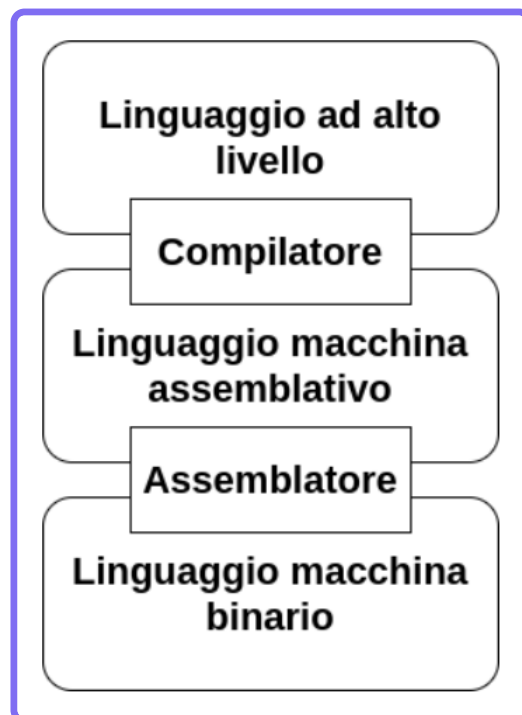


06_livello_software

Questo file è la rielaborazione delle slide [06_Livello-software.pdf](#)

Il processo di programmazione consiste nel seguente paradigma:

1. Il programmatore scrive il programma in **linguaggio ad alto livello**
2. Il **compilatore** traduce il linguaggio ad alto in **linguaggio macchina assembleativo**
 - trasforma una serie di file sorgenti di linguaggio ad alto livello in file sorgenti assembly
3. L'**assemblatore** traduce il programma in linguaggio assembleativo in **sequenze binarie**
 - trasforma una serie di file sorgenti assembly in file oggetto
4. Il **linker** collega assieme vari file oggetto e file di libreria in un unico programma oggetto



Per generare il file oggetto l'assemblatore esegue i seguenti passi:

1. Genera la codifica delle istruzioni espresse in assembly
2. Riconosce le direttive di assemblatore per l'allocazione di memoria mettendo queste informazioni nell'header del file oggetto

3. Riconosce le direttive che assegnano nomi a costanti
4. Sostituisce il valore binario ad ogni occorrenza di etichette di indirizzi relativi nel codice

Nel caso in cui l'assemblatore fosse del tipo "a due passi", man mano che incontra le etichette e dichiarazione di costanti, tiene traccia dei nomi dentro una **tabella dei simboli**, per poi sostituire ogni occorrenza di un nome con il valore indicato nella tabella. Questo tipo di assemblatore viene detto a 2 passate perché fa tutto ciò in 2 passi distinti:

- Passo 1: scorre il codice e raccoglie tutti i simboli
 - Passo 2: scorre di nuovo il codice sostituendo i simboli con i valori in tabella
-

Loader

File sorgente, file oggetto e dati sono memorizzati nella memoria secondaria, per essere caricati in memoria facciamo uso del loader, per fare ciò esegue i seguenti step:

1. Leggere le informazioni quali la lunghezza del programma e la locazione dell'header del file oggetto.
 2. Caricare il programma in memoria sulla base di tali informazioni
 3. Saltare alla prima istruzione del programma da eseguire
-

Linker

Nella maggior parte dei casi un programma è distribuito in più file sorgente, in questi casi l'assemblatore genera un file oggetto incompleto, il file oggetto completo lo genera il **linker** che si occupa di combinare più file oggetto separati risolvendo i riferimenti a nomi esterni, inoltre crea il file eseguibile e lo salva sul disco.

Librerie

i file sorgente esterni di cui parlavamo nel linker, molte volte fanno parte di librerie di file, ovvero degli archivi di file oggetto utilizzabili in programmi esterni, i riferimenti vengono risolti sempre dal linker che andrà a risolvere i vari riferimenti nel programma oggetto finale.

Compilatore

Questo programma si occupa di trasformare un file sorgente scritto in linguaggio ad alto livello in un file scritto in assembly, un compilatore che riorganizza le istruzioni per ottimizzare il codice viene detto ottimizzante. Un programma ad alto livello può chiamare sottoprogrammi presenti in altri file assembly o scritti in altri linguaggi (il linker gestirà i collegamenti). Il compilatore è in grado di rilevare errori sintattici e nomi sconosciuti nel codice sorgente, ma non errori di programmazione (bug)

Debugger

il debugger è un programma che ci permette di eseguire il programma oggetto ed interrompere la sua esecuzione in qualsiasi istante, in modo da poterne valutare il suo funzionamento. Esistono 2 tipi di debugger:

- **Trace mode:** il programma viene eseguito passo-passo, interrompendosi dopo ogni istruzione
 - Si genera un'eccezione al termine dell'esecuzione di ogni istruzione del programma, il debugger viene lanciato come routine di servizio dell'istruzione cosicché il programmatore possa controllare il corretto funzionamento di quell'istruzione, una volta che il programmatore seleziona il comando per continuare l'esecuzione viene effettuato un rientro dall'interruzione e viene eseguita l'istruzione successiva
- **breakpoint:** l'esecuzione del programma si interrompe in punti di osservazione specifici
 - Quando il Debugger è in esecuzione, il programmatore può scegliere dei punti di osservazione (breakpoint) dove interrompere il programma. il debugger sostituisce e mette da parte le istruzioni in corrispondenza dei breakpoint con speciali interruzioni software (Trap), il programma viene eseguito normalmente fino ad arrivare alla prima Trap, dove l'esecuzione passa al Debugger, una volta che il programmatore seleziona il comando per continuare l'esecuzione il Debugger riprende l'esecuzione del programma

Sistema operativo

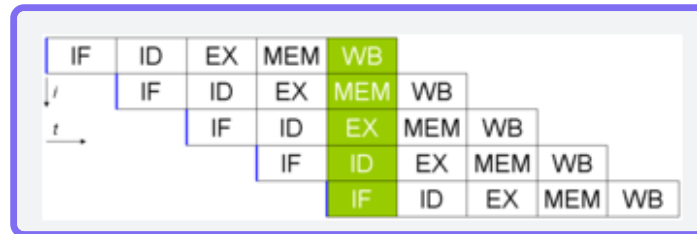
Il sistema operativo gestisce il coordinamento generale di tutte le attività del calcolatore. Il SO è formato da un insieme di routine essenziali che risiedono nella memoria centrale e un insieme di programmi di utilità che risiedono su disco e vengono caricati in memoria centrale per essere eseguiti. Durante l'inizializzazione del sistema, un processo di avvio (boot-strapping) viene usato per caricare in memoria una porzione iniziale del SO. Sistemi operativi capaci di eseguire più istruzioni contemporaneamente sono chiamati **concorrenti** o **multitasking**, per fare ciò il sistema operativo divide il tempo di esecuzione di un programma in slice di tempo, sarà lo scheduler a scegliere quale slice eseguire. I programmi possono trovarsi in 3 stati: **running**, **runnable** e **blocked**

07_Pipelining

Questo file è la rielaborazione delle slide [07_Pipelining.pdf](#)

Per svolgere compiti sempre più complessi in tempi ridotti è stata introdotta nei processori la tecnologia del pipelining (idea presa dalle catene di montaggio delle fabbriche) l'obiettivo di questa tecnica è quello di parallelizzare quante più istruzioni possibili, ricordando la suddivisione delle istruzioni in 5 fasi diverse (Prelievo – Decodifica –

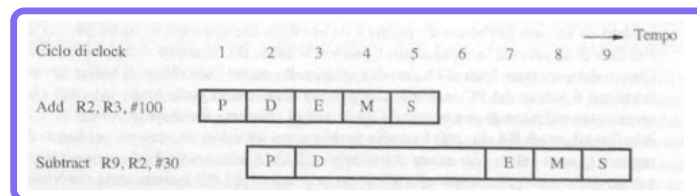
Elaborazione – Memoria - Scrittura) nel caso migliore avremo 5 istruzioni eseguite in parallelo. Per gestire l'esecuzione in pipeline di più istruzioni abbiamo bisogno di mantenere le istruzioni tra uno stadio e l'altro, queste istruzioni vengono conservate dai **buffer interstadio**. Di seguito una pipeline perfetta dove ad un certo punto vengono eseguite 5 istruzioni contemporaneamente.



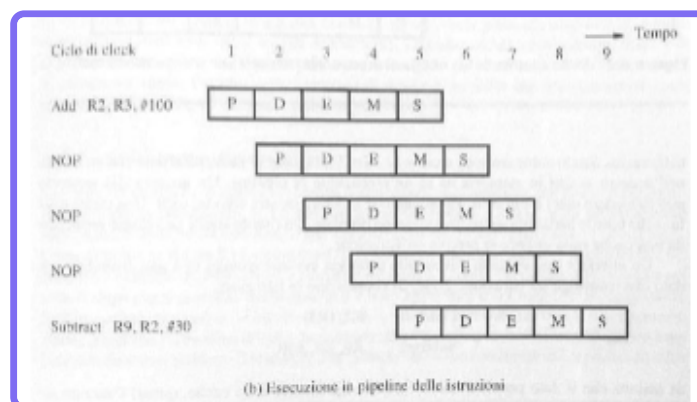
Non è sempre possibile avere la situazione ideale infatti possono avvenire delle problematiche di seguito discusse che rallentano l'esecuzione della pipeline.

Data Hazard

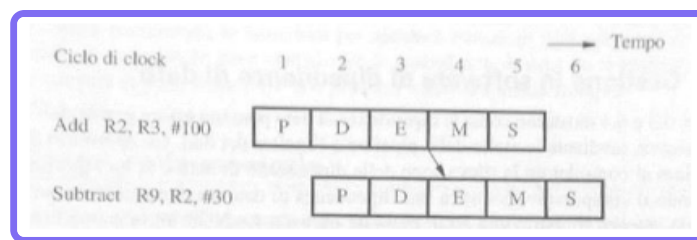
Questo tipo di conflitto avviene quando un'istruzione richiede un dato da un registro e quel dato non è stato ancora aggiornato dalle istruzioni precedenti.



Date le due istruzioni in foto capiamo che l'istruzione **Subtract** dovrà aspettare che l'istruzione **Add** abbia finito, questo allunga inevitabilmente la durata della pipeline perché **Subtract** resta in stallo aspettando l'aggiornamento del registro **R2**, per porre un'istruzione in stallo vengono generate delle istruzioni nulle (**NOP**) che sostanzialmente creano un ciclo di inattività da parte del processore, queste istruzioni vengono generate dal compilatore o via hardware, di seguito le istruzioni dell'esempio precedente ma con l'introduzione di istruzioni NOP



In questo modo l'istruzione Subtract inizia solo quando Add ha finito. Per evitare cicli della CPU a vuoto (ovvero le istruzioni NOP) si fa uso della tecnica dell'**operand forwarding**, il risultato viene salvato nei registri interstadio sin dalla fase di execute in questo modo l'istruzione che ha bisogno di quel dato non deve aspettare che l'istruzione dalla quale dipende finisca tutte le fasi ma basta che arrivi alla fase di execute.



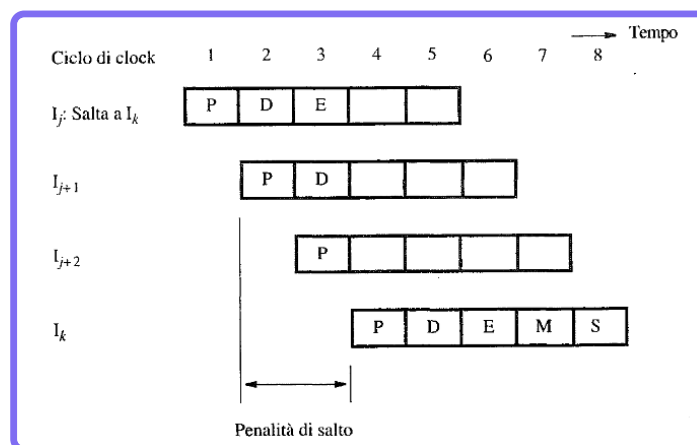
altre dipendenze del dato affrontate nei capitoli precedenti

Ritardi nell'accesso alla memoria

Gli accessi alla memoria alcune volte necessitano di diversi cicli di clock infatti nel caso un dato/istruzione non si trovi nella **cache** si possono avere ritardi di 10 o più cicli di clock che sarebbe il tempo che ci mette il processore a prendere i dati necessari dalla RAM, questo crea chiaramente dei grandi ritardi nella pipeline.

Control hazards

Si verifica un problema nella pipeline quando vengono eseguiti **salti condizionali**. Durante l'elaborazione di un'istruzione di salto, l'indirizzo di destinazione viene caricato nel **Program Counter (PC)** solo al **terzo stadio della pipeline** (fase di esecuzione). Di conseguenza, le istruzioni già caricate negli stadi successivi della pipeline, che si trovano dopo il salto, risultano errate e devono essere scartate



Per mitigare l'impatto dei salti condizionali e migliorare l'efficienza, si possono adottare diverse strategie:

- **Salto in fase di decodifica:** si può modificare l'hardware in modo da effettuare il salto direttamente nello stadio di decodifica
- **Tecnica di salto differito:** dove le istruzioni successive al salto vengono eseguite in ogni caso, sarà il compilatore a modificare l'ordine di esecuzione delle istruzioni in modo da eseguire delle istruzioni che vanno eseguite a prescindere negli slice saltati, se non trova istruzioni valide inserisce delle NOP.

Branch penalties

I salti vengono chiamati **Branch**, i ritardi che accadono a causa dei salti vengono chiamati **Branch Delays** oppure **Branch Penalties** questi problemi vengono risolti attraverso le **Branch Prediction**, se la predizione non va a buon fine abbiamo un **Branch Misprediction** quando accada questa cosa viene avviato un **flush** della pipeline dopo la quale la pipeline verrà riempita nel modo giusto. Esistono 2 tipi di branch prediction:

- **Predizione statica:** usa regole semplici basate su informazioni statiche come la posizione dell'istruzione nel codice o il tipo di istruzione di salto, questo approccio statico è molto rapido e semplice ma può essere meno preciso in situazioni complesse.
- **Predizione dinamica:** una tecnica che si basa sulla storia dei branch eseguiti in precedenza. Questo approccio sfrutta i **buffer di branch prediction** (BPB), che fungono da memoria storica per migliorare l'accuratezza della predizione. Più grande è il buffer, maggiore è la precisione della predizione. Ogni branch analizzato viene associato a uno stato, che varia a seconda dell'architettura della macchina:
 - **In alcune macchine**, gli stati possibili sono due:
 - **PS** (Probabilmente Salta)
 - **PNS** (Probabilmente Non Salta)
 - **In altre macchine**, il modello può essere più complesso e utilizzare quattro stati:
 - **MPS** (Molto Probabilmente Salta)
 - **PS** (Probabilmente Salta)
 - **PNS** (Probabilmente Non Salta)
 - **MPNS** (Molto Probabilmente Non Salta)

Questo sistema consente di migliorare significativamente la predizione, adattandosi dinamicamente al comportamento dei branch.

Per poter iniziare a fare la predizione dalla fase di Fetch si ha bisogno di una memoria piccola e veloce chiamata **Buffer di destinazione di salto** questo buffer contiene una tabella con tutte le istruzioni di salto del programma. Per ogni istruzione saranno salvate le seguenti informazioni:

- Indirizzo dell'istruzione
- Uno o due bit di stato per l'algoritmo di predizione
- Indirizzo di destinazione del salto

Usando questa tabella ogni volta che viene prelevata un'istruzione, il suo indirizzo verrà cercato nella tabella, se l'istruzione prelevata è un salto si useranno le informazioni in tabella per aggiornare il program counter. In grandi programmi la tabella non contiene tutte le istruzioni di salto, ma viene aggiornata man mano.

Structural Hazards

La pipeline va in stallo quando una risorsa viene richiesta da più istruzioni, per evitare questo problema l'unica soluzione è quella di avere cache separate per istruzione e dati.

Valutazione delle prestazioni

La valutazione delle prestazioni di un processore viene fatta sia per i processori che usano la pipeline sia per quelli che non la usano, la formula chiaramente cambia. Di seguito la legenda per capire le formule successive:

- N : Numero di istruzioni macchina
- S : Numero di cicli di clock per istruzione (CPI - *Cycles Per Instruction*)
- T_{clock} : Durata di un ciclo di clock ($T_{clock} = \frac{1}{R}$)
- R : frequenza di clock del processore

💡 Tip

La formula per calcolare T (tempo di esecuzione) di N istruzioni in un processore con pipeline è la seguente: $T = \frac{N \cdot S}{R}$

💡 Tip

La formula per calcolare il throughput di un processore è la seguente:

Senza pipeline: $P_{np} = \frac{R}{S \cdot T_{clock}}$

Con pipeline: $P_p = \frac{R}{S}$

nel caso ottimale con pipeline il throughput è uguale ad R

Come già detto la pipeline soffre di conflitti, prendendo in considerazione questi conflitti la formula per il calcolo del throughput diventa:

💡 Formula calcolo del throughput

$$P_p = \frac{R}{S + \sigma_{dato} + \sigma_{salto} + \sigma_{miss}}$$

Ogni tipologia di conflitto indipendente aumenta S di un delta δ dato dal numero di occorrenze del conflitto p per il numero medio di cicli di stallo introdotti c per evitarlo:

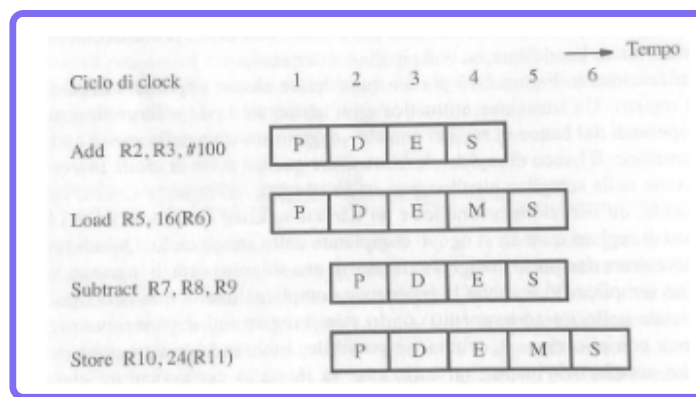
- Conflitti di dipendenza di dato: $\sigma_{dato} = p_{dato} \cdot c_{dato}$
- Conflitti di salto: $\sigma_{salto} = p_{salto} \cdot c_{salto}$
- Conflitti di cache miss: $\sigma_{miss} = p_{miss} \cdot c_{miss}$

Processori superscalari

I processori con più unità di elaborazione vengono chiamati **superscalari**, nel caso di un processore con 2 unità di elaborazione:

- **Unità aritmetica:** esegue le istruzioni aritmetico-logiche
- **Unità Load/Store:** esegue le istruzioni di accesso alla memoria

Nel seguente caso la pipeline di questo tipo di processore è diversa da quella di un normale processore, in specifico cambia così:



Le istruzioni aritmetiche e di accesso alla memoria possono essere eseguite in parallelo a coppie e quindi nei primi due cicli di clock si possono mandare in esecuzione le quattro istruzioni. All'entrata di ogni unità di esecuzione troviamo una **stazione di prenotazione** dove sono presenti:

- Tutte le istruzioni in attesa di esecuzione
 - Informazioni e operandi rilevanti per ogni istruzione che troviamo
- Un'istruzione viene mandata in esecuzione solo quando tutti i suoi operandi sono disponibili

Fase di smistamento nei processori superscalari

Nella **fase di smistamento** il processore deve assicurarsi che tutte le risorse necessarie ad un'istruzione siano disponibili, in specifico si occupa di verificare la disponibilità dei registri temporanei per contenere i risultati, che ci sia abbastanza spazio nella stazione di prenotazione dell'unità di esecuzione desiderata, e che ci sia una locazione disponibile nel buffer di riordino, inoltre si occupa di prevenire i **deadlock** (casi in cui due istruzioni rimangono bloccate a causa di dipendenze reciproche)

Esecuzione fuori ordine nei processori superscalari

L'**esecuzione fuori ordine** nei processori superscalari migliora l'efficienza eseguendo istruzioni non appena pronte, ma può causare conflitti o eccezioni imprecise. Per evitare errori, i risultati vengono salvati in registri temporanei tramite il **register renaming**. L'**unità di commitment**, supportata dal **buffer di riordino**, trasferisce poi i risultati ai registri permanenti rispettando l'ordine originale delle istruzioni, garantendo coerenza e correttezza nell'esecuzione.

Pipeline con i processori CISC

I processori **CISC** hanno diverse difficoltà nell'utilizzare le pipeline a causa della complessità delle istruzioni, per risolvere questo problema quasi tutti i processori odierni sono basati

su CISC ma le istruzioni vengono dinamicamente convertite in micro-istruzioni RISC che posso essere eseguite nella pipeline in modo più agevole.

08_Struttura-base-del-processore

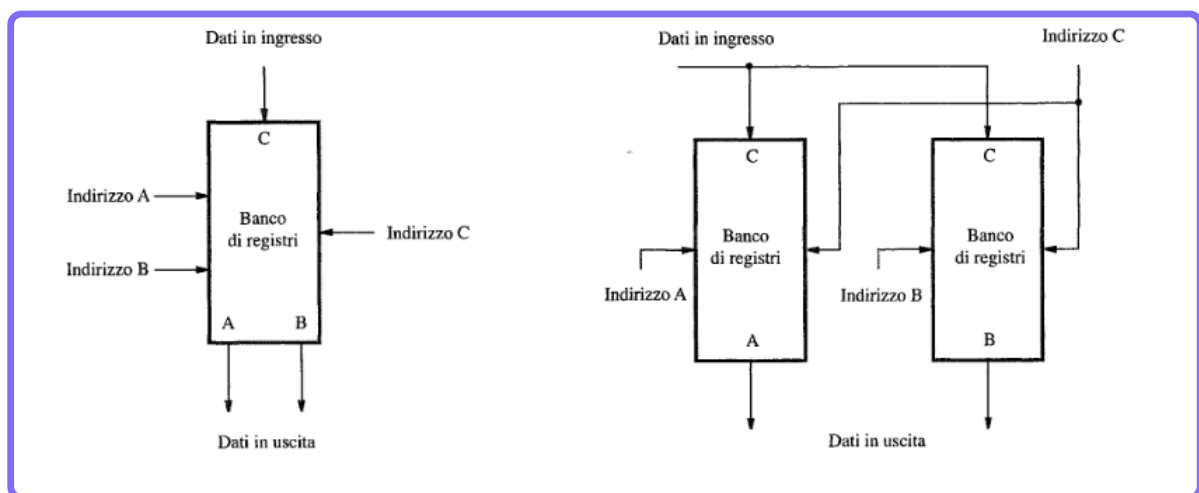
Questo file è la rielaborazione delle slide [08_Struttura-base-del-processore.pdf](#)

La CPU è un circuito elettronico integrato che ha il ruolo di cervello del calcolatore, e capace di eseguire istruzioni elementari necessarie per eseguire i programmi. È formato da diversi componenti:

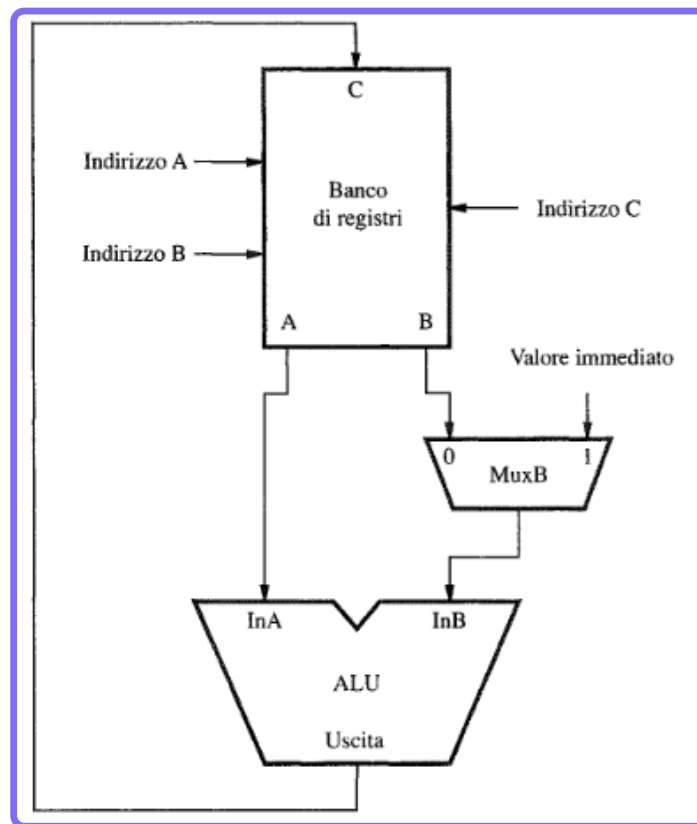
- La **ALU** (Unità logico-aritmetica): esegue le operazioni aritmetiche e logiche necessarie ad eseguire un programma
- **CU** (Control Unit): genera i bit di controllo per gestire il funzionamento della CPU
- **Banco di registri**: blocchi di memoria usati dalla CPU durante l'esecuzione delle istruzioni
 - **PC** (Program counter): un registro che contiene l'indirizzo della prossima istruzione da eseguire
 - **IR** (Instruction Register): un registro che contiene l'indirizzo dell'istruzione in esecuzione
- **Generatore di indirizzi**: un componente che si occupa di aggiornare il contenuto del PC
- **Interfaccia processore memoria**: gestisce il trasferimento dei dati tra memoria e CPU

Banco di registri

Un blocco di memoria piccolo e veloce, consiste in vari registri con un circuito per l'accesso in scrittura e lettura. È possibile leggere contemporaneamente i dati da due registri diversi, invece la lettura può avvenire da un registro alla volta, per selezionare il registro da leggere o scrivere, si utilizzano specifici ingressi di indirizzo.



Un componente della CPU che esegue operazioni aritmetiche e logiche quali la somma, sottrazione, AND, OR, XOR, ecc. É formato da 2 porte di input che rappresentano gli operandi in ingresso collegate direttamente al banco dei registri, una porta di uscita contenente il risultato dell'operazione anche questa collegata al banco dei registri. Alla ALU viene collegato anche il MUX (un multiplexer) che è in grado di introdurre un altro valore all'interno della ALU



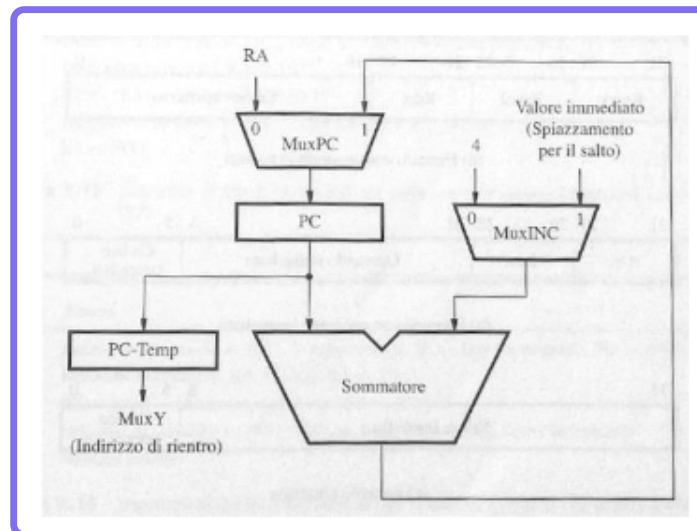
Link register

un registro essenziale per la gestione delle chiamate a sottoprogrammi. Quando un programma chiama una funzione o una procedura, l'indirizzo dell'istruzione successiva alla chiamata viene salvato nel link register. Una volta terminata l'esecuzione della sotto procedura, il valore contenuto nel link register viene caricato nel program counter (PC) per far riprendere l'esecuzione del programma principale dal punto esatto in cui era stato interrotto. **Ad esempio**, se un programma principale chiama una funzione per calcolare il fattoriale di un numero, il link register conterrà l'indirizzo dell'istruzione successiva alla chiamata. Al termine del calcolo del fattoriale, il programma tornerà a eseguire l'istruzione successiva alla chiamata iniziale.

Generatore di indirizzi delle istruzioni

Questo circuito è usato per generare l'indirizzo della prossima istruzione da inserire nel PC. In questo componente troviamo un sommatore che incrementa il valore del PC di una word solitamente (4 byte) o anche di più in caso di salto. Il tipo di incremento da effettuare verrà fatto dal MuxINC, il MUXPC seleziona se aggiornare il PC con l'incremento calcolato dal

MuxINC o con un indirizzo specifico, inoltre abbiamo PC-Temp che viene usato per salvare il valore di PC da inserire nel [LR](#) durante una chiamata a sottoprogramma.



Temporizzazione del processore

Per il corretto funzionamento di un processore, è necessario un segnale elettrico periodico che scandisca il tempo al suo interno, comunemente noto come **segnale di clock**. Questo segnale, simile a un metronomo, coordina le operazioni dei vari componenti, indicando con precisione quando eseguire ciascuna operazione. Il clock ha il compito fondamentale di sincronizzare il flusso dei dati all'interno del processore, garantendo che ogni componente riceva ed elabori i dati al momento opportuno. Ogni operazione del processore avviene all'interno di un intervallo di tempo definito come **ciclo di clock**, che rappresenta la durata di un singolo impulso del segnale di clock. Durante un ciclo di clock, i dati vengono trasferiti, elaborati e memorizzati, con ogni fase dell'operazione che si svolge in maniera sequenziale e coordinata. La velocità del processore, comunemente espressa in Hertz (Hz), misura il numero di cicli di clock completati in un secondo, determinando così la rapidità con cui il processore può eseguire le istruzioni.

Esecuzione delle istruzioni

Il processore per mettere in esecuzione un'istruzione segue i seguenti step:

1. Prelievo dell'istruzione della memoria (all'indirizzo puntato da PC)
2. Incremento di PC di una word (per farlo puntare all'istruzione successiva)
3. Esecuzione dell'istruzione prelevata

I primi due passi compongono la **fase di prelievo** e il terzo la **fase di esecuzione**

≡ Esempio con delle istruzioni assembly:

- **LOAD R5, X(R7)**
questa istruzione carica il valore del registro con indirizzo $R7 * X$ in R5 e viene eseguita in questo modo:

1. **Prelievo dell'istruzione ed incremento del PC:** Il processore legge l'istruzione dalla memoria e incrementa il Program Counter (PC) per puntare all'istruzione successiva.
 2. **Decodifica dell'istruzione e lettura del contenuto del registro R7:** Il processore decodifica l'istruzione per capire quale operazione deve eseguire e legge il valore contenuto nel registro R7.
 3. **Calcolo dell'indirizzo effettivo:** Il processore somma il valore contenuto in R7 con l'offset X per ottenere l'indirizzo di memoria esatto da cui leggere il dato.
 4. **Lettura dell'operando sorgente dalla memoria:** Il processore accede alla memoria all'indirizzo calcolato al punto precedente e legge il valore che trova in quella locazione.
 5. **Caricamento dell'operando nel registro di destinazione R5:** Il valore letto dalla memoria viene copiato nel registro R5, sovrascrivendo qualsiasi valore precedentemente presente in quel registro.
- ADD R3, R4, R5
 1. Prelievo dell'istruzione ed incremento del PC
 2. Decodifica dell'istruzione e lettura dei contenuti dei registri sorgenti R4 e R5
 3. Calcolo della somma $R4 + R5$
 4. Caricamento del risultato nel registro di destinazione R3

Più in generale qualsiasi istruzione può essere eseguita in 5 stadi distinti:

1. **Fetch:** preleva un'istruzione e incrementa il contatore del programma
2. **Decode:** decodifica l'istruzione e legge i registri dal banco dei registri
3. **Execute:** esegui un'operazione dell'ALU
4. **Memory:** leggi o scrivi dati in memoria
5. **Write back:** il risultato nel registro di destinazione

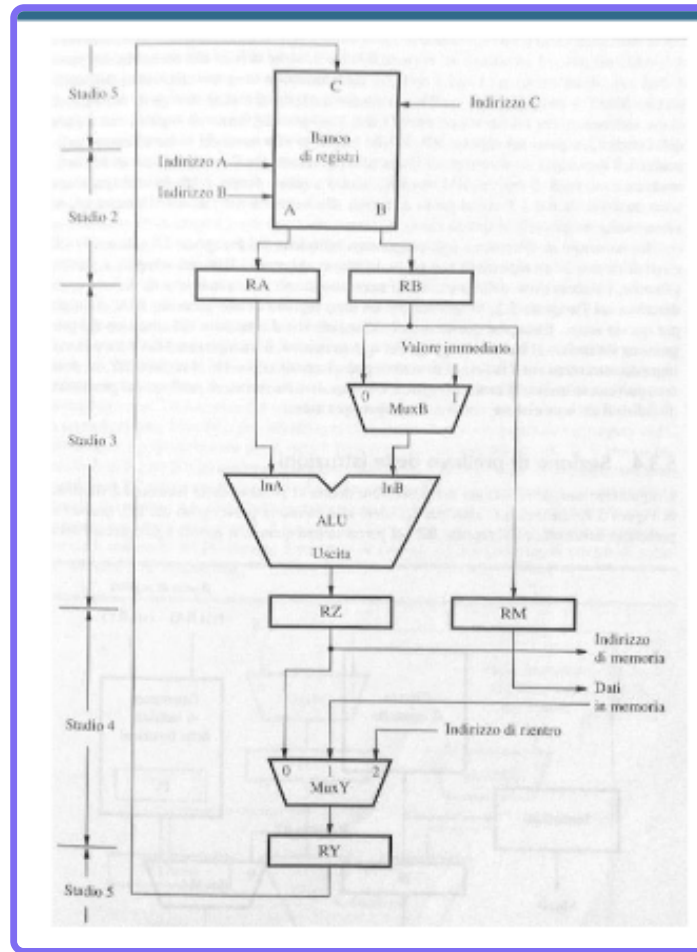
Calcolo in un solo stadio:

- **Svantaggi:** Se si volesse eseguire un'intera operazione in un singolo ciclo di clock, il ciclo dovrebbe essere molto lungo. Questo perché il circuito dovrebbe avere il tempo sufficiente per completare tutte le fasi dell'operazione, dalla lettura dei dati, all'esecuzione dei calcoli, fino alla scrittura del risultato.
- **Limiti di performance:** Un ciclo di clock lungo limita la frequenza di funzionamento del processore, riducendone le prestazioni.

Divisione in più stadi:

- **Vantaggio:** Dividendo l'esecuzione in più stadi più semplici, è possibile accorciare la durata di ciascun ciclo di clock.
- **Pipeline:** con l'introduzione della pipeline otteniamo dei grandi vantaggi dalla parallelizzazione dei singoli stadi.
- **Registri temporanei:** Tra uno stadio e l'altro vengono inseriti dei registri temporanei che memorizzano i dati intermedi, consentendo il passaggio fluido da uno stadio all'altro.

Questa è la rappresentazione grafica del data path in modo approfondito ([qui la spiegazione semplificata](#))



Quello che vediamo in questa immagine può essere riassunto in questo modo:

1. Stadio 1 (non visibile in foto)

in questo stadio il datapath si occupa di prelevare i dati dalla memoria, in questo stadio troviamo un multiplexer che decide se prendere l'indirizzo di memoria o da un registro oppure dal PC, fatto ciò mette decodifica l'istruzione e la mette in esecuzione.

2. Stadio 2:

- **Testo:** Le porte di uscita A e B del banco di registri copiano i dati nei registri temporanei RA e RB.
- **Nel diagramma:**
 - Il banco di registri fornisce i valori richiesti dalle porte A e B.
 - Questi valori vengono copiati rispettivamente nei registri RA e RB.

3. Stadio 3:

- **Testo:**
 - RA viene usato come primo ingresso dell'ALU.
 - MuxB decide se il secondo ingresso dell'ALU proviene da RB o da un valore immediato.
 - Il risultato dell'ALU è copiato in RZ.
 - Il valore di RB è copiato in RM.
- **Nel diagramma:**
 - RA è connesso direttamente al primo ingresso dell'ALU.

- MuxB seleziona tra RB e un valore immediato per il secondo ingresso dell'ALU.
- Il risultato dell'ALU (RZ) viene memorizzato.
- Parallelamente, il contenuto di RB è inviato al registro temporaneo RM.

4. Stadio 4:

- **Testo:**
 - Se necessario, l'indirizzo contenuto in RZ è inviato all'interfaccia del processore con la memoria.
 - Se richiesto, i dati in RM sono salvati in memoria.
 - MuxY seleziona il valore da memorizzare in RY: può essere il risultato dell'ALU, un dato dalla memoria, o un indirizzo di ritorno da subroutine.
- **Nel diagramma:**
 - RZ può essere utilizzato come indirizzo per accedere alla memoria.
 - I dati temporanei in RM possono essere scritti in memoria, se necessario.
 - Il MuxY decide quale valore salvare in RY.

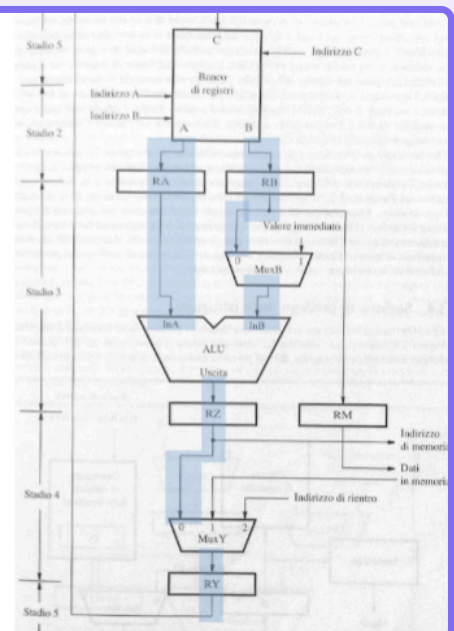
5. Stadio 5:

- **Testo:** Il contenuto di RY viene salvato nel banco di registri.
- **Nel diagramma:**
 - L'uscita di RY è connessa alla porta di ingresso C del banco di registri.
 - Il dato è quindi memorizzato nel registro corrispondente.

Esempio

Add R3, R4, R5

1. Indirizzo di memoria $\leftarrow$ [PC], Leggi memoria, $IR \leftarrow$ Dati da memoria, $PC \leftarrow [PC] + 4$
2. Decodifica istruzione, $RA \leftarrow [R4]$, $RB \leftarrow [R5]$
3. $RZ \leftarrow [RA] + [RB]$
4. $RY \leftarrow [RZ]$
5. $R3 \leftarrow [RY]$



Segnali del processore

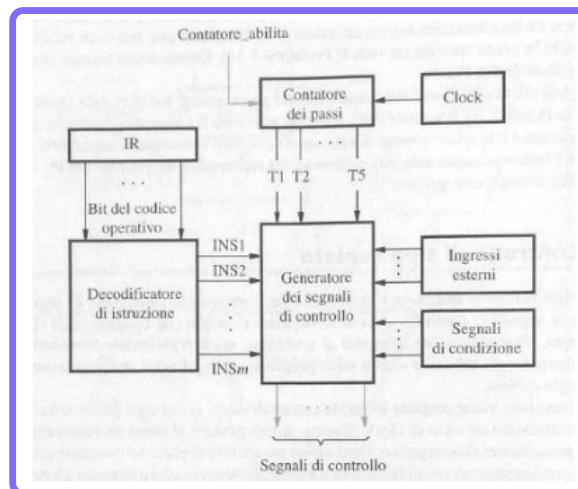
Per eseguire le istruzioni macchina il processore deve generare le sequenze di segnali di controllo per ogni stadio, questi segnali si dividono in:

- Segnali di selezione per i multiplatori

- Segnali di attivazione di alcuni registri
- Segnali di condizione
- Segnali per la gestione della memoria

Un approccio per generare i segnali di controllo consiste nel **controllo cablato** formato dai seguenti componenti:

- **Contatore dei passi:** che scandisce gli stadi di esecuzione, questo contatore all'inizio dell'esecuzione di un'istruzione ha come valore 0 che aumenta fino a 5 per ogni stadio di esecuzione
- **Decodificatore di istruzione:** genera un vettore lungo **m** mettendo a 1 solo il bit corrispondente all'istruzione letta su **IR**
- **Generatore dei segnali di controllo:** produce i segnali di controllo sulla base dell'istruzione in esecuzione, dello stadio attuale (letto dal contatore), dei segnali di condizione e di segnali esterni come le interruzioni. I segnali generati sono predefiniti a livello hardware



Non sempre gli accessi alla memoria possono essere eseguiti in un singolo ciclo di clock, infatti se una risorsa non si trova dentro la cache (cache miss) ci vogliono molteplici cicli di clock per recuperarla, quando questo succede la control unit dei processori si mette in ascolto aspettando un segnale chiamato **MFC (memory function completed)** che indica il corretto caricamento in memoria di un dato, è importante ricordare che durante l'accesso alla memoria il contatore dei passi deve essere bloccato per fare ciò viene generato un segnale **WMFC (Wait for memory function completed)** che dice al contatore dei passi di aspettare un segnale MFC (e quindi la fine della fase di fetch) per riprendere a contare.

- $AbilitaContatore = \neg WMFC + MFC$

⚡ Danger

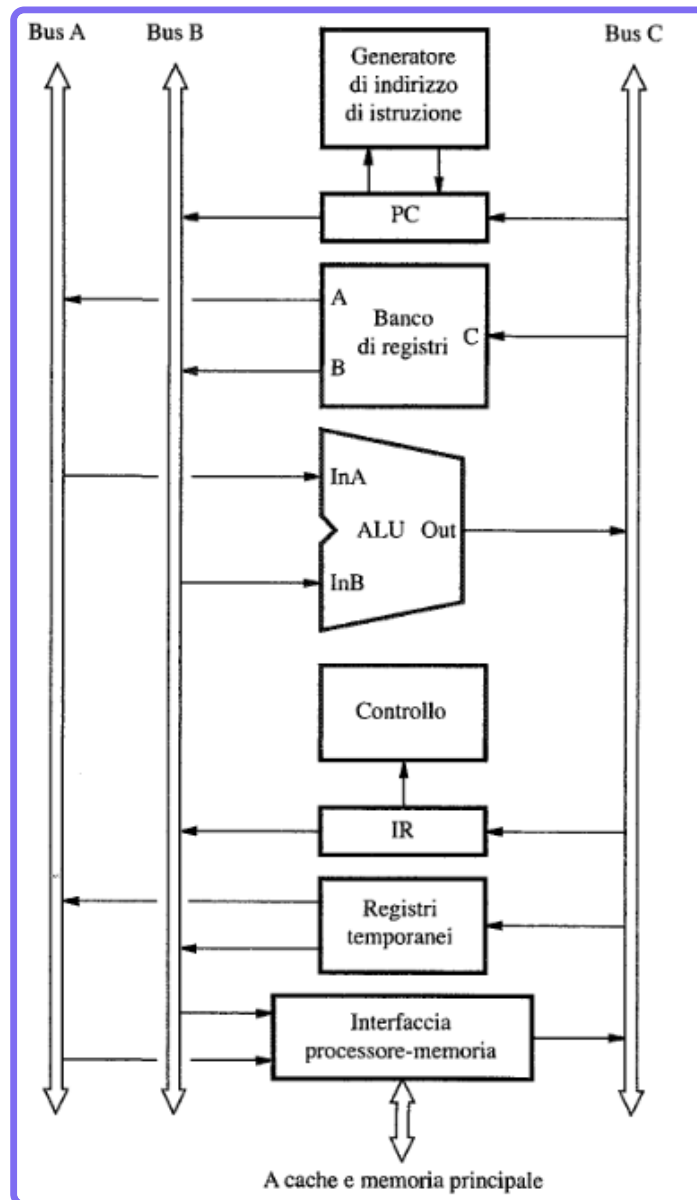
Tutto quello che abbiamo descritto fino ad adesso vale solo per i processori **RISC** infatti quest'ultimi sono gestibili in più stadi grazie alla lunghezza delle istruzioni ridotta.

Interconnessione

All'interno di un processore tutti i componenti vengono messi in contatto attraverso il blocco di interconnessione, quest'ultimo è un insieme di bus. La porta logica che invia un segnale su una linea di bus è chiamata **bus driver**, i dispositivi sono collegati al bus tramite le **porte tri-state**, a differenza delle normali porte logiche che hanno solo due stati (0 e 1), le porte tri-state possono assumere un terzo stato: l'alta impedenza. In pratica, quando una porta è in questo stato, si comporta come se fosse scollegata dal circuito, non influenzando il segnale presente sul bus. Il bus verrà influenzato solo dalle porte non in alta impedenza.

Interconnessione usando 3 bus

Il blocco di interconnessione può essere costruito usando 3 BUS il bus A, e il B usati per i dati in input e il Bus C per i dati in output, il generatore di indirizzi è collegato direttamente al PC. Con questo tipo di architettura possiamo eseguire un'istruzione in 3 cicli di clock.

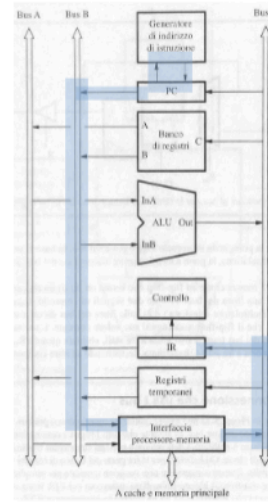


Primo ciclo:

Add R5, R6

.L'istruzione di addizione fra due registri può essere eseguita in 3 passi:

1. Indirizzo di memoria $\leftarrow [PC]$, Leggi memoria, Attesa MFC, $IR \leftarrow$ Dati da memoria, $PC \leftarrow [PC] + 4$



Secondo ciclo:

Esempio Add via registri

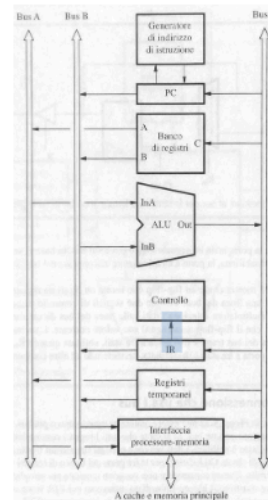


Add R5, R6

.L'istruzione di addizione fra due registri può essere eseguita in 3 passi:

1. Indirizzo di memoria $\leftarrow [PC]$, Leggi memoria, Attesa MFC, $IR \leftarrow$ Dati da memoria, $PC \leftarrow [PC] + 4$

2. Decodifica Istruzione



Terzo ciclo:

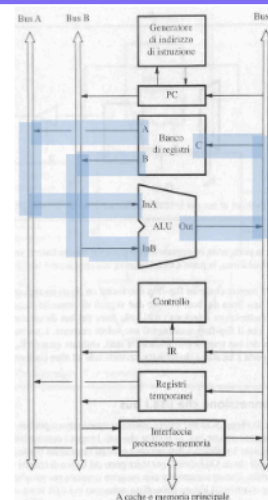
Add R5, R6

.L'istruzione di addizione fra due registri può essere eseguita in 3 passi:

1. Indirizzo di memoria $\leftarrow [PC]$, Leggi memoria, Attesa MFC, $IR \leftarrow$ Dati da memoria, $PC \leftarrow [PC] + 4$

2. Decodifica Istruzione

3. $R5 \leftarrow [R5] + [R6]$



Controllo microprogrammato

I segnali di controllo di ogni passo vengono raccolti in una word di memoria chiamata **microistruzione**. L'insieme di microistruzioni rappresentanti i passi di un'istruzione macchina si chiamano **microroutine**. Le microistruzioni di ciascuna microroutine vengono immagazzinate in locazioni consecutive della **memoria di controllo**, il registro μPC contiene l'istruzione della prossima microistruzione da caricare. All'inizio di un'istruzione macchina il generatore di indirizzi delle microistruzioni carica sul μPC la prima istruzione della microroutine corrispondente ad ogni passo μPC viene incrementato di un passo per puntare alla microistruzione corretta.

Microprogrammato vs Microcablato

Controllo microcablato:

- **Implementazione:** I segnali di controllo sono generati attraverso un circuito logico cablato.
- **Velocità:** È generalmente più veloce poiché non dipende dall'accesso a una memoria di controllo.
- **Flessibilità:** È meno flessibile, perché le modifiche al set di istruzioni richiedono un aggiornamento fisico dell'hardware.
- **Complessità:** La complessità cresce rapidamente con il numero di istruzioni e segnali di controllo.

Controllo microprogrammato:

- **Implementazione:** I segnali di controllo sono memorizzati in una memoria di controllo sotto forma di "microistruzioni" organizzate in "microroutine".
- **Velocità:** Più lento rispetto al microcablato a causa dei tempi di accesso alla memoria di controllo.
- **Flessibilità:** Molto più flessibile, perché le modifiche al set di istruzioni possono essere apportate cambiando le microistruzioni nella memoria.
- **Complessità:** È più scalabile, particolarmente utile per architetture complesse.

È difficile determinare quale sia la migliore infatti questo dipende dall'applicazione:

- Il **microcablato** è preferibile in sistemi con requisiti di alte prestazioni e dove il set di istruzioni è stabile, come nelle applicazioni embedded o nei processori ad alta velocità.
- Il **microprogrammato** è ideale per processori generici o complessi (ad esempio, con molte istruzioni) dove è necessaria flessibilità, come nei primi sistemi CISC.

09_Algebra-booleana

Questo file è la rielaborazione delle slide [09_Algebra-booleana.pdf](#)

Operatori logici

l'algebra booleana della commutazione è un sistema algebrico in cui ogni variabile può assumere solo 2 valori (0 o 1). Possiede 3 operazioni basilari:

- **Somma logica o OR**

- ovvero una funzione che accetta 2 argomenti è vale 1 solo se almeno uno dei suoi ingressi è 1. Si denota tramite gli operatori a due argomenti “+” o “V”.
- Di seguito le proprietà:
 - **Commutativa:** $x_1 + x_2 = x_2 + x_1$
 - **Associativa:** $x_1 + (x_1 + x_2) = (x_1 + x_2) + x_3$
 - **Elemento neutro:** $0 + x = x$

- **Prodotto logico o AND**

- Una funzione che accetta 2 argomenti in input e vale 1 solo se tutti i suoi ingressi sono 1. Si denota tramite gli operatori a due argomenti “.” o “^”
- Di seguito le proprietà:
 - **Commutativa:** $x_1 \cdot x_2 = x_2 \cdot x_1$
 - **Associativa:** $x_1 \cdot (x_1 \cdot x_2) = (x_1 \cdot x_2) \cdot x_3$
 - **Elemento neutro:** $1 \cdot x = x$

- **Complementazione o NOT**

- è una funzione che accetta un'argomento e ne inverte il valore. Si denota tramite l'operatore di sopra lineatura “~” o di negazione “¬”

Di seguito la tabella di verità di tutte le espressioni

A	B	A OR B	A AND B	NOT A
0	0	0	0	1
0	1	1	0	1
1	0	1	0	0
1	1	1	1	0

OR (somma) e AND (prodotto) hanno le seguenti proprietà:

SOMMA	Proprietà distributiva:	PRODOTTO
$x + y \cdot z = (x + y) \cdot (x + z)$		$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$
Proprietà di idempotenza:		
$x + x = x$		$x \cdot x = x$
Proprietà di complemento:		
$x + \overline{x} = 1$		$x \cdot \overline{x} = 0$
Proprietà dello 1 e dello 0:		
$1 + x = 1$		$0 \cdot x = 0$

Varie definizioni

Funzione logica: definiamo funzione logica una funzione con più variabili binarie di ingresso ed una variabile binaria di uscita.

Tabella di verità: tutte le funzioni logiche possono essere espresse attraverso una tavola di verità, queste tabelle sono formate da 2^n righe ed $n + 1$ colonne, dove n è il numero di variabili in ingresso.

Espressioni logiche: Unendo più funzioni logiche si ottengono le espressioni logiche, esistono infinite espressioni logiche che danno come risultato la stessa funzione logica. Due espressioni logiche si dicono equivalenti se rappresentano la stessa funzione logica, come nell'esempio di seguito:

$$(x_1 + x_2)(x_1 + \overline{x_2})(\overline{x_1} + x_2) = x_1 x_2$$

x_1	x_2	$x_1 \cdot x_2$	$(x_1 + x_2)(x_1 + \neg x_2)(\neg x_1 + x_2)$
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

Nel risolvere un'espressione logica dobbiamo stare attenti alla precedenza tra gli operatori, di seguito gli ordini di precedenza da seguire:

Operatore	Precedenza
Negazione - NOT	1
Prodotto - AND	2
Somma - OR	3

Tip

Per calcolare i valori assunti da una funzione ci basta calcolare la sua tabella di verità

Mintermine: funzione ad n variabili che vale 1 solo per una specifica configurazione delle variabili.

Maxtermine: funzione ad n variabili che vale 0 solo per una specifica configurazione delle variabili.

Forme canoniche: tutte le espressioni logiche possono essere rappresentate in delle forme dette "[normali](#)"

Forma minima: Un'espressione logica si dice in forma minima quando non esiste nessun'altra espressione equivalente con un costo inferiore, il costo di un'espressione logica è dato dal **numero di comparse delle variabili** nell'espressione stessa.

$$(x_1 + x_2)(x_1 + \overline{x_2})(\overline{x_1} + x_2) = x_1 x_2$$

Costo 6 Costo 2

per passare da prima forma canonica a forma minima seguire i seguenti passi:

.Usando la **proprietà distributiva**, associare le coppie di mintermini che posseggono una sola variabile in forma discordante (diretta e negata)

$$\overline{x_1}\overline{x_2}\overline{x_3} + \overline{x_1}\overline{x_2}x_3 = \overline{x_1}\overline{x_2}(\overline{x_3} + x_3)$$

.Usare la **legge di complemento** per trasformare in 1 le somme di variabili complementari

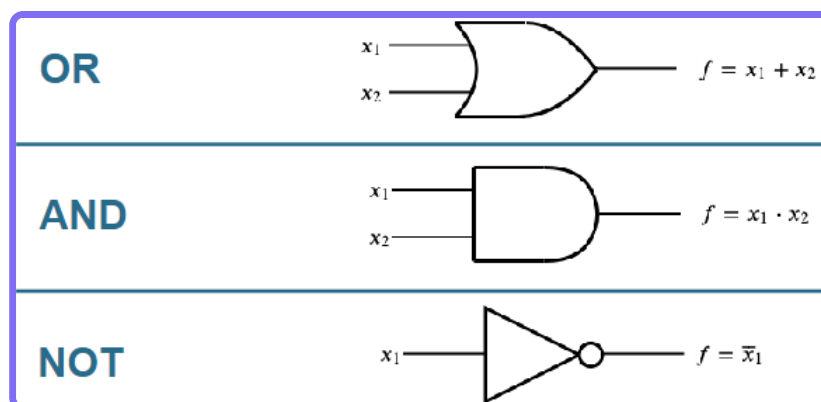
$$\overline{x_1}\overline{x_2}(\overline{x_3} + x_3) = \overline{x_1}\overline{x_2}$$

.Usare la **legge di idempotenza** per duplicare dei mintermini nel caso fosse necessario

$$x + x = x$$

Porte logiche

Tutte le operazioni logiche citate possono essere eseguite da semplici circuiti elettronici, questo circuiti sono formati da delle porte logiche, ognuna va a rappresentare una delle operazioni logiche sopracitate.



Queste porte godono di tutte le proprietà delle funzioni logiche. Inoltre abbiamo anche altre 3 porte molto importanti:

1. **XOR** (anche detto OR esclusivo): funzione che vale 1 solo se gli 1 nei sono ingressi sono in numero dispari, si denota con il simbolo " $\oplus$ "

Porta EXOR



$$Y = A \oplus B$$

Ingresso A	Ingresso B	Uscita Y
0	0	0
0	1	1
1	0	1
1	1	0

2. **NAND**: porta che si denota con il simbolo " $\uparrow$ " e ha la seguente tabella di verità

Porta NAND



$$Y = \overline{A \cdot B}$$

Ingresso A	Ingresso B	Uscita Y
0	0	1
0	1	1
1	0	1
1	1	0

3. **NOR**: porta che si denota con il simbolo " $\downarrow$ " e ha la seguente tabella di verità

Porta NOR

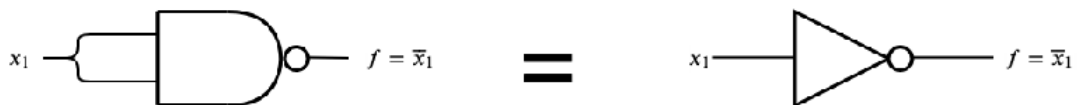


$$Y = \overline{A + B}$$

Ingresso A	Ingresso B	Uscita Y
0	0	0
0	1	1
1	0	1
1	1	0

Le porte NAND e NOR sono considerate porte universali, usando queste porte si può realizzare qualsiasi funzione.

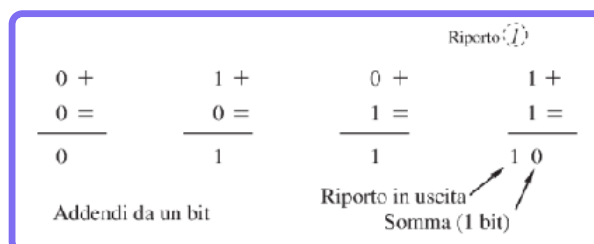
- una porta NAND con ingressi unificati si comporta come una porta NOT



10_Circuiti-aritmetici

Questo file è la rielaborazione delle slide [10_Circuiti-aritmetici.pdf](#)

La somma tra due numeri binari avviene in questo modo:



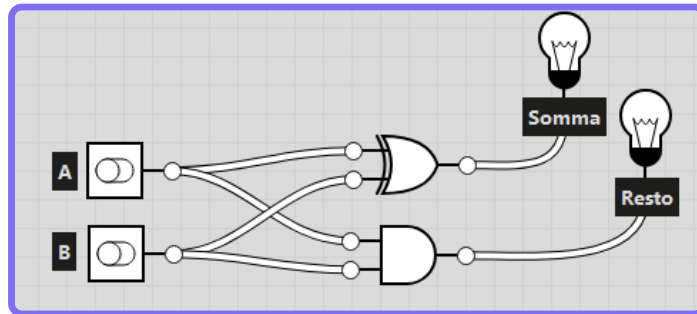
Il **riporto in uscita** della somma precedente viene assegnato come **riporto in entrata** alla

somma successiva. Questo tipo di addizioni vengono fatte continuamente all'interno della nostra macchina, e questo viene fatto attraverso gli addizionatori, esistono vari tipi di addizionatore:

- **Half adder:** Un circuito molto semplice che fa la somma di 2 bit (calcolando il riporto in uscita) senza considerare alcun riporto in ingresso. Ha 2 uscite:

- **Somma:** $S = A \oplus B$ ovvero l'uscita di una porta XOR
- **Riporto:** $C = A \cdot B$ ovvero l'uscita di una porta AND

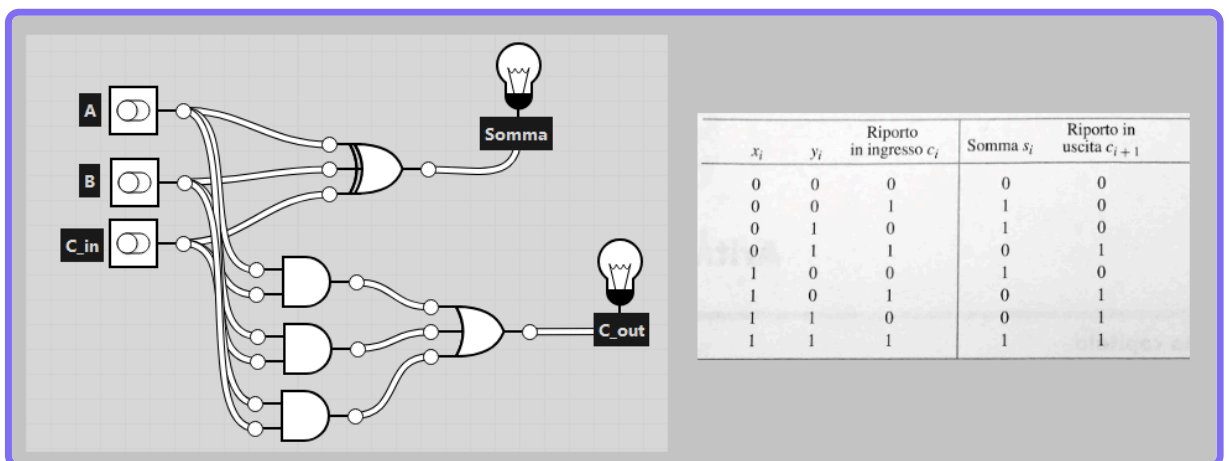
Di seguito la rappresentazione grafica di questo circuito:



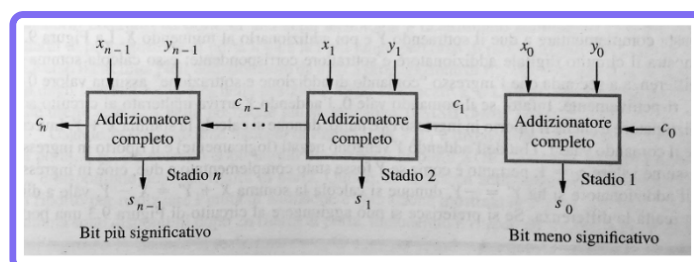
- **Full adder:** anche chiamato **addizionatore ad 1 bit** può essere espresso da 2 funzioni logiche a tre ingressi, i due bit da sommare (chiamati in seguito A e B) e l'eventuale riporto in entrata (chiamato in seguito C_{in}):

- **Somma:** calcola la somma tra i 2 bit in entrata ed il riporto in ingresso, usando questa formula: $S = A \oplus B \oplus C_{in}$ praticamente una porta XOR a tre ingressi.
- **Riporto:** La seconda funzione logica calcola il riporto in uscita, usando questa formula: $C_{out} = (x_i \times c_i) + (y_i \times c_i) + (x_i \times y_i)$

Di seguito la tabella di verità e la rappresentazione grafica del circuito:

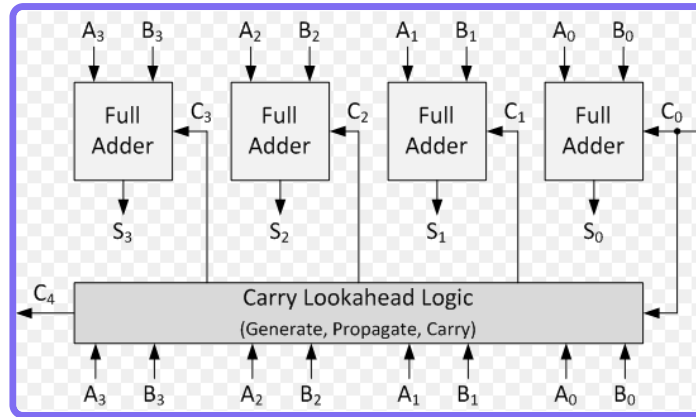


- **Ripple carry adder:** anche chiamato addizionatore con propagazione di riporto, si ottiene collegando tra di loro diversi full-adder in modo da propagare il riporto, di seguito la rappresentazione grafica:



Il problema principale di questo addizionatore è che ogni full adder per andare avanti deve aspettare il calcolo del resto dell'adder precedente, questo crea dei rallentamenti. Il termine "ripple" indica che il **riporto** si propaga sequenzialmente da un bit al successivo, come un'onda.

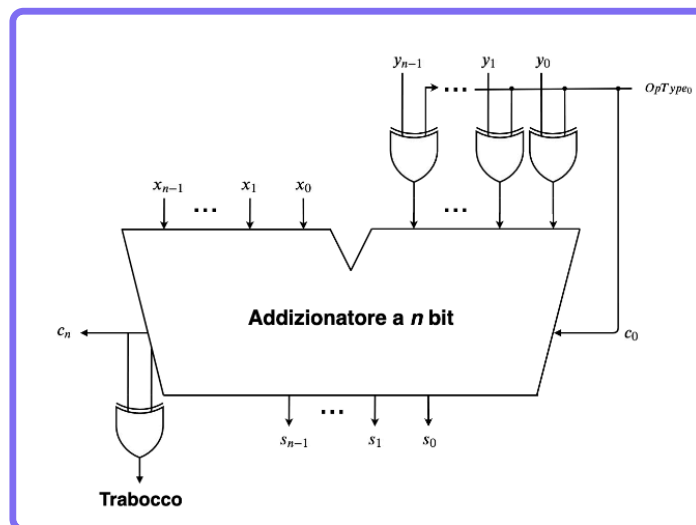
- **Carry lookahead adder:** per risolvere il problema del ripple carry adder nasce questo tipo di addizionatore, che va a calcolare il resto in parallelo propagandolo in base a delle formule matematiche (funzioni G_i e P_i spiegato in seguito), tutto ciò lo rende più veloce e quindi viene ancora oggi usato dentro le ALU dei processori.



Trabocco (Overflow)

Si parla di trabocco quando andiamo a sommare dei bit e il risultato dell'operazione aritmetica è un numero che supera le capacità di rappresentazione del circuito. Il trabocco avviene molto spesso nelle operazioni algebriche dove si prendono in considerazione numeri con segno.

- **Addizionatore algebrico:** è un circuito logico che somma due numeri binari a n bit, considerando anche il segno, la parte importante per questo circuito è la gestione dell'Overflow, infatti viene calcolato utilizzando questa formula: $c_n \oplus c_{n-1}$ con c_n che sarebbe il riporto generato dall'ultimo bit, e c_{n-1} il riporto generato dal penultimo bit, in pratica se il risultato di questa formula ci dà 1 c'è stato un trabocco.



Ripple Carry Adder e Calcolo del Ritardo

Il ritardo totale del circuito dipende dal **percorso più lento**.

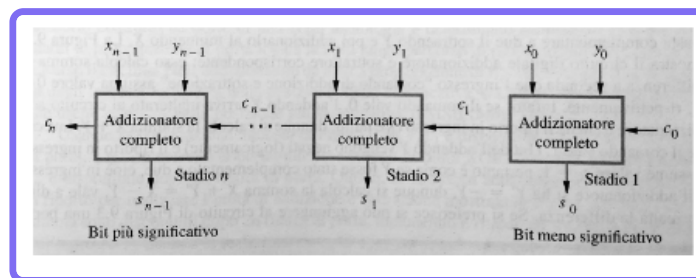
- In un **Full Adder (FA)**:
 - La somma parziale è calcolata dopo **1 ritardo di porta**:
$$s_i = x_i \oplus y_i \oplus c_i$$
 - Il riporto è calcolato dopo **2 ritardi di porta**:
$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$
- In un **Ripple Carry Adder** a n bit:
 - L'ultimo riporto è calcolato dopo $2n$ ritardi di porta.
 - L'ultimo bit di somma è calcolato dopo $2n - 1$ ritardi di porta.

Funzioni di Generazione e Propagazione

Riscrivendo: $c_{i+1} = x_i y_i + (x_i + y_i) c_i$ introduciamo:

- **Funzione di generazione**: $G_i = x_i y_i$
- **Funzione di propagazione**: $P_i = x_i + y_i$

Il riporto diventa: $c_{i+1} = G_i + P_i c_i$ questa formula ci avvantaggia perché G_i e P_i dipendono solo dagli ingressi x_i e y_i e possono essere calcolati **in parallelo**.



Carry lookahead adder a 2 livelli: Per ridurre ulteriormente il tempo di calcolo si utilizza una struttura a più livelli logici, in pratica si prende la struttura di Carry lookahead adder e la si estende su due livelli.

Circuito moltiplicatore sequenziale: gli addizionatori vengono anche utilizzati per fare le moltiplicazioni, infatti la moltiplicazione di numeri senza segno può essere realizzata usando un **addizionatore ad n bit** e **due registri**.

```
A = 0
```

```
Q = Moltiplicatore
```

```
M = Moltiplicando
```

```
Per n cicli:
```

```
    se q[0] = 1:
```

```
        A = A + M
```

altrimenti:

$$A = A + 0$$

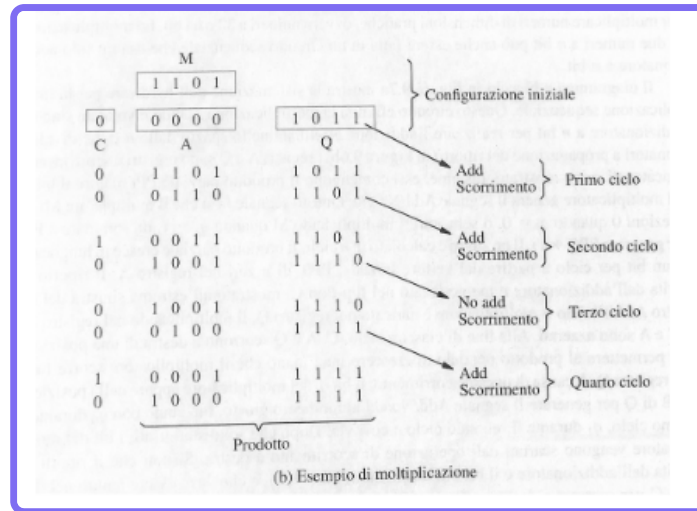
c = riporto

A scorre verso destra di una posizione

Q scorre verso destra di una posizione

c scorre verso destra di una posizione

Alla fine del ciclo concatenando i contenuti di A e Q si otterrà il prodotto finale



Moltiplicazione dei numeri con segno: per i numeri con il segno la situazione si complica un po', usiamo l'**algoritmo di Booth**: L'algoritmo di Booth è un metodo efficiente per moltiplicare numeri binari con segno in complemento a due. L'idea chiave è ricodificare il moltiplicatore come somme e sottrazioni di potenze di 2. Nel caso semplice in cui il moltiplicatore contiene una sequenza contigua di 1, Il prodotto è uguale al moltiplicando fatto scorrere di 5 posizioni a sinistra + il complemento a 2 del moltiplicando fatto scorrere di 1 posizione a sinistra.

Ricodifica bit-pair

Il ricodifica Bit-pair è una tecnica di compressione dei dati utilizzata per ridurre la lunghezza di una sequenza di bit senza perdere informazioni questo lo fa raggruppando i bit in coppia (bit pair) e li codifica in un nuovo formato.

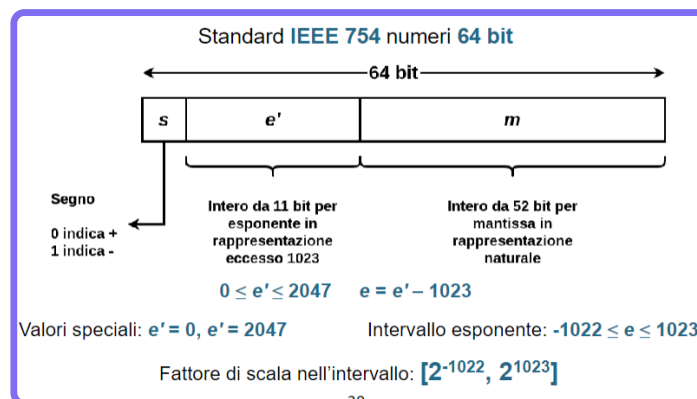
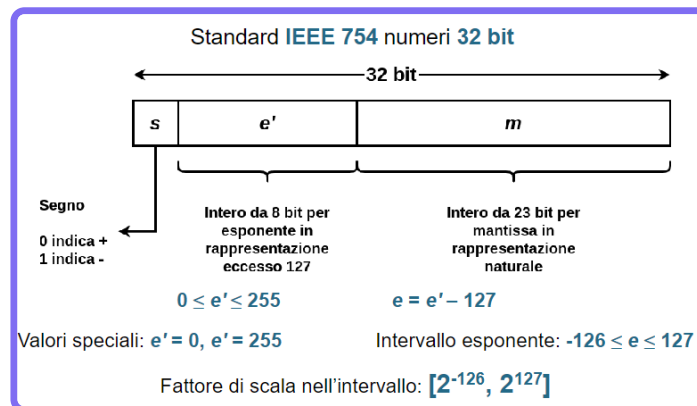
Divisione con ripristino: anche questo tipo di circuito viene realizzato usando un addizionatore n+1 bit, un registro(M) e 2 shift register (A e Q), questi componenti vengono usati dentro un ciclo che esegue n volte i seguenti passaggi:

- Fare scorrere A e Q a sinistra di una posizione
- Sottrarre M da A e porre il risultato in A

- Se il segno di A è 1, porre q_0 a 0 e sommare M ad A ; altrimenti, porre q_0 a 1
All'inizio M contiene il Divisore, A contiene 0 e Q contiene il Dividendo
Alla fine A contiene il Resto e Q contiene il Quoziente

Numeri in virgola mobile: un numero in binario con virgola mobile viene rappresentato da:

- Un **segno** s per il numero
- La **mantissa** m
- Un **esponente** e con segno in base 2



11_Insieme-di-istruzioni-macchina

Questo file è la rielaborazione delle slide [11_Insieme-di-istruzioni-macchina.pdf](#)

ISA

Le istruzioni elementari che un processore è in grado di eseguire sono definite nell'ISA (Instruction Set Architecture), queste istruzioni elementari vengono definite attraverso un alfabeto binario. Il linguaggio assembly (**assembly**) è una rappresentazione simbolica del linguaggio macchina. Esistono 2 grandi famiglie di ISA:

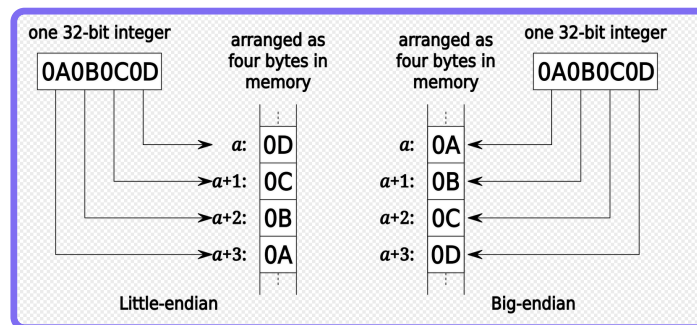
- **CISC** (Complex Instruction Set Architecture): Basata su istruzioni complesse, che possono anche occupare più di una word di memoria.

- **RISC**(Reduced Instruction Set Architecture): Basata su istruzioni semplici, con le istruzioni che al massimo occupano una word, di solito sono molto efficienti grazie all'utilizzo della [pipeline](#)

Memoria

Il calcolatore lavora su gruppi di bit detti **Parole**(word), ogni parola nella nostra memoria è associata ad un indirizzo binario univoco, parole consecutive sono associate ad indirizzi consecutivi. Un numero binario con m bit potrà indirizzare al massimo 2^m indirizzi. L'unità minima di informazione indirizzabile in memoria è il byte, di solito i dati vengono immagazzinati in 2 modi diversi:

- **Big-endian(Crescente)**: memorizzazione/trasmissione che inizia dal byte più significativo (estremità più grande) per finire col meno significativo
- **Little-endian(Decrescente)**: memorizzazione/trasmissione che inizia dal byte meno significativo (estremità più piccola) per finire col più significativo.



Programmare in assembly

Il programmatore scrive i programmi in **linguaggio assembler** (assembly), che vengono tradotti in linguaggio macchina dall'**assemblatore**. Il codice risultante in linguaggio macchina viene poi eseguito direttamente dal processore.

Registri e locazioni di memoria

All'interno del nostro processore abbiamo diverse locazioni di memoria usate per immagazzinare dati di lavoro usati dalla CPU durante l'esecuzione delle istruzioni. Di solito i registri generici del processore vanno da: R0 ad Rn, abbiamo anche dei registri speciali come il [PC](#) (Program Counter) e [IR](#) (Instruction Register)

Istruzioni di base in assembly:

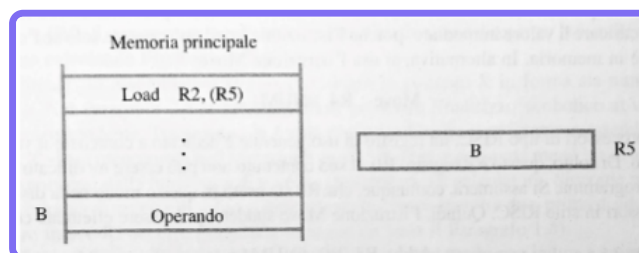
```
LOAD destinazione sorgente
STORE sorgente destinazione
ADD destinazione sorgente1 sorgente2
SUB destinazione sorgente1 sorgente2
```

Queste istruzioni fanno:

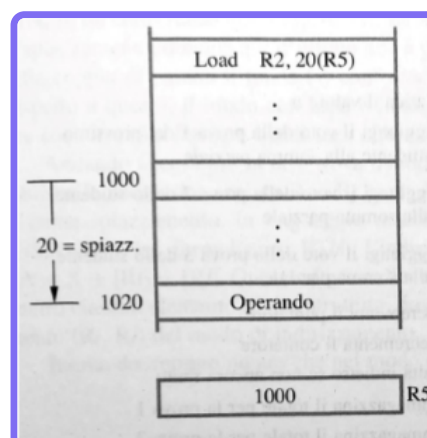
- **LOAD**: carica un dato dalla memoria in un registro del processore
- **STORE**: carica un dato presente in un registro di un processore in memoria
- **ADD**: somma il contenuto di 2 registri
- **SUB**: sottrae il contenuto di 2 registri

Nel linguaggio assembler, gli operandi e il risultato delle istruzioni possono essere espressi in modi diversi, i metodi con cui specificare operandi e risultato vengono chiamati modi di indirizzamento, i modi di indirizzamento base sono:

- **Modo immediato**: specificando direttamente l'operando
 - Add R4, R6, #200
- **Modo di registro**: indicando il nome del registro del processore
 - Load R2, R5
- **Modo assoluto**: indicando l'indirizzo di una word in memoria
 - Load R2, #0x5D
- **Indiretto da registro**: indicando un registro con all'interno l'indirizzo della word in memoria
 - Load R2,(R5)



- **Con indice e spiazzamento**: L'indirizzo effettivo di un operando è ottenuto prendendo l'indirizzo come nel caso "indiretto da registro" e sommandogli una costante
 - LOAD R2,20(R5)



- **Con base e indice**: l'indirizzo effettivo è ottenuto sommando il contenuto di 2 registri.

Istruzione condizionale:

```
Branch_if condizione destinazione salto.  
Branch_if_[R2]>0 CICLO
```

Istruzione usata per saltare all'esecuzione di un'istruzione specifica nel caso la condizione di salto sia vera

Direttive di assembler

Per aiutare l'assembler nella traduzione del nostro programma in codice binario esistono **le direttive all'assembler**, queste non vengono tradotte in vero e proprio codice binario ma servono per dare delle informazioni utili all'assembler. Ne esistono di vario tipo:

```
NOME EQU valore_numerico  
ORIGIN indirizzo_memoria  
RESERVE spazio_in_byte  
DATAWORD contenuto_da_assegnare
```

- **EQU**: associa il valore numerico nome
 - **ORIGIN**: indica l'indirizzo di partenza dove inserire le istruzioni
 - **RESERVE**: indica all'assembler di riservare uno spazio di memoria espresso in byte
 - **DATAWORD**: Inizializza una word di memoria con il contenuto passato
-

Generiche istruzioni assembly:

Di solito le istruzioni di assembly seguono la seguente struttura:

Linea di codice assembly



.Nel maggiore dei casi, una linea di codice assembly presenta i seguenti campi:

Etichetta **Operazione** **Operandi** **Commento**

.Etichetta: Nome che viene associato all'indirizzo della parola di memoria assegnata all'istruzione o all'indirizzo del blocco di memoria riservato. È facoltativa

.Operazione: Il nome (codice operativo) dell'istruzione oppure una direttiva di assembler.

.Operandi: Informazione di indirizzamento per accedere agli operandi

.Commento: Testo di commento, ignorato dall'assembler

Inoltre l'assemblatore ci permette di denotare i numeri in diversi formati:

- **Binaria:** %
 - Add R2, R3, #%01011101
- **Decimale:** nessun prefisso
 - Add R2, R3, #93
- **Esadecimale:** 0x
 - Add R2, R3, #0x5D

Pila(Stack)

Per gestire gli aspetti fondamentali di un programma (le chiamate a funzione, allocazione dinamica, ecc...) assembly usa lo stack, questa è una struttura dati basata sul paradigma **LIFO** (Last In First Out), abbiamo un registro speciale del processore chiamato **SP** (Stack pointer) che punta alla cima della lista, in questa struttura dati si possono fare solo 2 operazioni:

- **PUSH:** aggiunge un elemento in cima alla pila, di solito si implementa così:

```
Load Rj, (SP);prendo il valore dalla cima e lo salvo in Rj
Add SP, SP, #4;aumento l'indirizzo così da spostarmi alla nuova cima
```

- **POP:** preleva un elemento dalla cima, di solito si implementa così:

```
Load Rj, (SP)
Add SP, SP, #4
```

Funzioni

Una funzione anche detto sottoprogramma è una lista di istruzioni che eseguono un compito specifico e che possono essere richiamate in un qualsiasi momento durante l'esecuzione di un programma, la chiamata alla funzione viene fatta usando una funzione di salto detta **Call Instruction**, nella funzione invece per ritornare alla routine chiamate viene usata l'istruzione **Return Instruction**, l'indirizzo di rientro di una funzione viene salvato nel **Link Register**.

```
CALL Indirizzo
RETURN
```

- **CALL:** chiama la funzione specificata seguendo questi 2 passi:
 1. Salva il contenuto del registro **PC** nel **Link register**
 2. Salta all'indirizzo di destinazione indicato nell'istruzione di chiamata

- **RETURN:** salta all'indirizzo di rientro e lo fa salvando nel **PC** il contenuto del **Link register**

Per passare i parametri a queste funzioni ci sono 2 modi:

- **Tramite registri:** il numero di dati è limitato al numero di registri
- **Tramite lo stack:** il numero di dati è virtualmente illimitato

il blocco dello stack riservato ai sottoprogrammi si chiama **Stack Frame**, il **Frame Pointer** è il puntatore a questa area della memoria, dentro questa area dello stack troviamo molte informazioni come le variabili locali e i loro valori. Quando abbiamo delle chiamate a funzione annidate dobbiamo ricordare di salvare l'indirizzo del link register dentro la pila, in modo da sapere sempre dove ritornare. In alcune architetture questa cosa viene fatta in automatico.

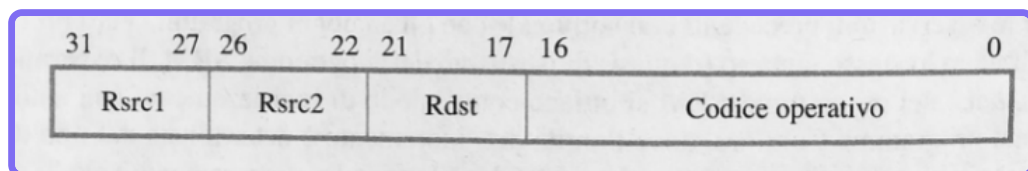
Codifica delle istruzioni

🔥 Codice operativo

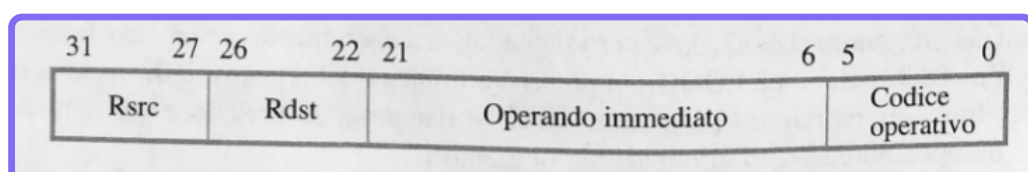
Il codice operativo o OpCode corrisponde alla procedura da mettere in esecuzione quando si deve eseguire un'istruzione

Le istruzioni in assembly possono cambiare in base all'architettura del processore, nel caso di una CPU a 32 bit possono essere usate le seguenti codifiche:

- **Formato con operandi in registri:** In questo caso parliamo di istruzioni di questo tipo:
ADD R1,R2,R3 dove:
 - I 15 bit più significativi vengono usati per gli indirizzi dei 3 registri (5 bit cadauno)
 - I restanti 17 bit per il codice operativo



- **Formato con operando immediato:** in questo caso parliamo di istruzioni di questo tipo:
ADD R1,R2,#5 dove:
 - 10 bit vengono usati per gli indirizzi dei registri,
 - 6 bit vengono usati per il codice operativo
 - 16 bit vengono usati per operando immediato



- **Formato per chiamata:** in questo caso parliamo di istruzioni di questo tipo: CALL 0x00400010 dove:

- 6 bit vengono usati per il codice operativo
- 26 bit per rappresentare l'indirizzo della funzione da chiamare

Register Transfer Notation (RTN)

La notazione a trasferimento di registro serve a descrivere formalmente il trasferimento di informazioni tra memoria e registri. Il simbolo utilizzato è $\leftarrow$:

- A sinistra troviamo un indirizzo di memoria
- A destra troviamo un valore semplice o un'espressione

.Esempio somma del contenuto di due registri (A e B). Nel registro C viene memorizzata la somma:

$$C \leftarrow [A] + [B]$$

Istruzioni logiche

AND destinazione, sorgente1, sorgente2

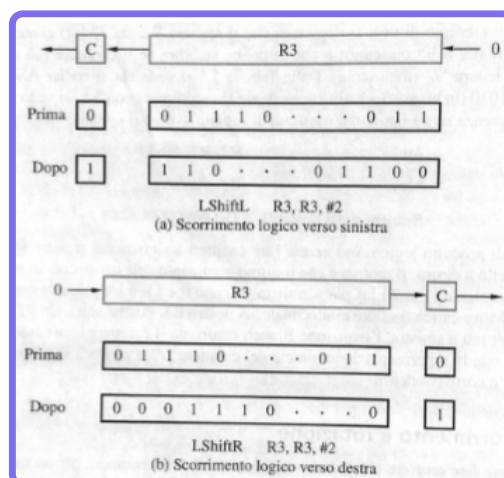
OR destinazione, sorgente1, sorgente2

- **AND**: fa un and tra le sorgenti
- **OR**: fa un or tra le sorgenti

Scorrimenti e rotazioni

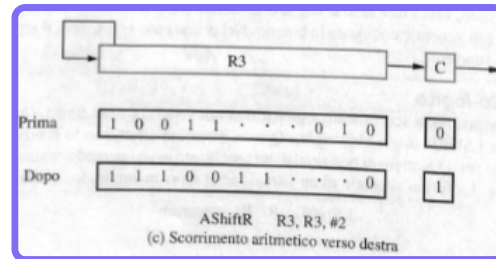
Scorrimento logico:

Le operazioni di scorrimento logico fanno scorrere i bit di un registro a destra o a sinistra di n posizioni, i bit in uscita vengono persi, ad eccezione dell'ultimo bit che viene memorizzato nel bit di riporto c



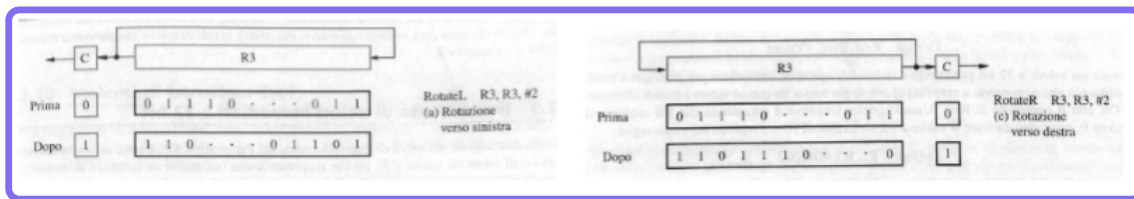
Scorrimento aritmetico:

Lo scorrimento aritmetico verso destra funziona come quello logico, ma riempie le posizioni lasciate libere con il valore del bit più significativo



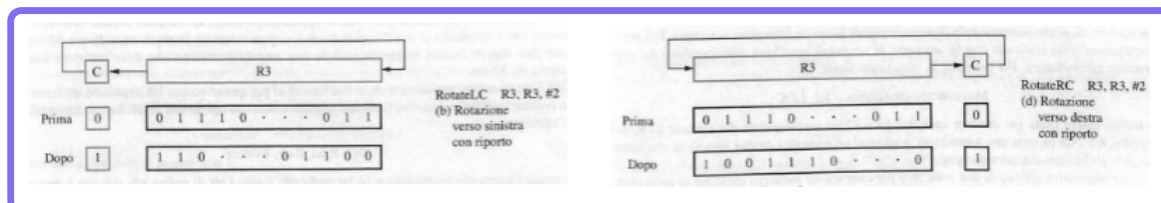
Rotazione

La rotazione è un'operazione di scorrimento dove i bit in uscita da un lato vengono fatti rientrare dall'altro. L'ultimo bit in uscita viene scritto nel bit di riporto c.



Rotazione con riporto

Nella rotazione con riporto il bit di riporto viene incluso nella sequenza di bit da ruotare. Nel caso di rotazione a sinistra il bit di riporto viene aggiunto alla sinistra dei bit del registro da ruotare. Nel caso di rotazione a destra il bit di riporto viene aggiunto alla destra dei bit del registro da ruotare



Altre istruzioni dell'assembly

```
LoadByte Rdst, LOCBYTE
StoreByte Rsrc, LOCBYTE
Multiply Rk, Ri, Rj
Divide Rk, Ri, Rj
```

- **LoadByte:** legge un singolo byte dalla memoria e lo registra negli 8 bit meno significativi del registro destinazione mettendo a 0 gli altri bit
- **StoreByte:** salva gli 8 bit meno significativi del registro sorgente nella locazione di memoria specificata
- **Multiply:** effettua la moltiplicazione tra i due numeri contenuti nei registri
- **Divide:** effettua la divisione tra due numeri in complemento a due contenuti in due registri

Istruzioni esclusive delle architetture CISC

Differenze tra stili RISC e CISC

RISC:

- Modi di indirizzamento semplici
- Meno istruzioni, tutte occupanti una singola parola
- Operazioni aritmetiche e logiche solo su registri
- Non sono possibili trasferimenti diretti tra due locazioni di memoria
- Possibile l'elaborazione a stadi
- Programmi di dimensioni maggiori

CISC:

- Modi di indirizzamento complessi
- Tante istruzioni complesse, occupanti più parole di memoria
- Operazioni aritmetiche e logiche con operandi sia su registri che locazioni di memoria
- Trasferimenti diretti tra due locazioni di memoria tramite istruzione Move
- Programmi di dimensioni ridotte

⚡ Danger

Le istruzioni scritte di seguito sono istruzioni esclusive di un architettura CISC

Istruzioni aritmetiche

Questo tipo di istruzioni hanno un formato a 2 indirizzi, dove l'indirizzo di destinazione servirà sia da ingresso che anche da risultato.

```
Operazione destinazione, sorgente  
ADD B,A
```

MOVE

```
Move destinazione, sorgente
```

L'istruzione Move permette il trasferimento di dati tra registri ed indirizzi di memoria

Indirizzamento per autoincremento

Nell'indirizzamento per autoincremento, l'indirizzo dell'operando è contenuto in un registro il cui nome viene specificato nell'istruzione. Alla fine dell'istruzione il contenuto del registro viene incrementato di un'unità. Può essere usata per eseguire l'operazione di Pop:

```
Move ELEMENTO, (SP)+
```

Indirizzamento per autodecremento

Nell'indirizzamento per autodecremento, l'indirizzo dell'operando è contenuto in un registro il cui nome viene specificato nell'istruzione. Prima di eseguire l'istruzione, il contenuto del registro viene decrementato di un'unità. Può essere usata per eseguire l'operazione di Push:

Bit di esito

sono bit speciali immagazzinati in un registro interno al processore, e vengono usati per tenere traccia dell'esito di svariate operazioni. Quelli più comuni sono:

Bit di esito	Significato
N (negativo)	1 se risultato negativo, 0 se positivo o nullo
Z (zero)	1 se risultato nullo, 0 altrimenti
V (trabocco)	1 se trabocco in comp. a due, 0 altrimenti (oVerflow)
C (riporto)	1 se trabocco in binario naturale, 0 altrimenti (Carry)

12_Tecnologia-microelettronica

Questo file è la rielaborazione delle slide [12_Tecnologia-microelettronica.pdf](#)

Rappresentazione delle variabili binarie

- Nei circuiti elettronici, i valori binari 0 e 1 sono rappresentati da livelli di tensione.
- La soglia separa gli 0 dagli 1. Ad esempio, in un sistema a 5V:
 - Tensioni sotto 1,5V = 0.
 - Tensioni sopra 3,5V = 1.
- Per evitare errori, esiste una "banda vietata" vicino alla soglia dove i valori non sono validi.
- La banda vietata riduce le incertezze causate da rumore e disturbi.

Cos'è un transistor?

- Un transistor è un semiconduttore usato per amplificare o commutare segnali elettronici.
- Materiali come il silicio sono drogati per creare zone positive e negative.
- Funziona come un interruttore che può essere in conduzione o in interdizione.
- Esistono due tipi principali di transistor:
 - **BJT (bipolare)**: La corrente è controllata da una corrente di base.
 - **FET (a effetto di campo)**: La corrente è controllata dalla tensione applicata al gate.

Tecnologia MOS (Metallo-Ossido-Semiconduttore)

- I transistor MOS sono i più comuni nei circuiti integrati.
- Valori tipici di tensione:

- $V_{cc} = 5V$, $V_{soglia} = 2,5V$
- $V_{cc} = 3,3V$, $V_{soglia} = 1,5V$
- Sopra la soglia = 1, sotto la soglia = 0.
- La tecnologia MOS consente di ridurre dimensioni e consumi, rendendo possibile l'integrazione di miliardi di transistor.

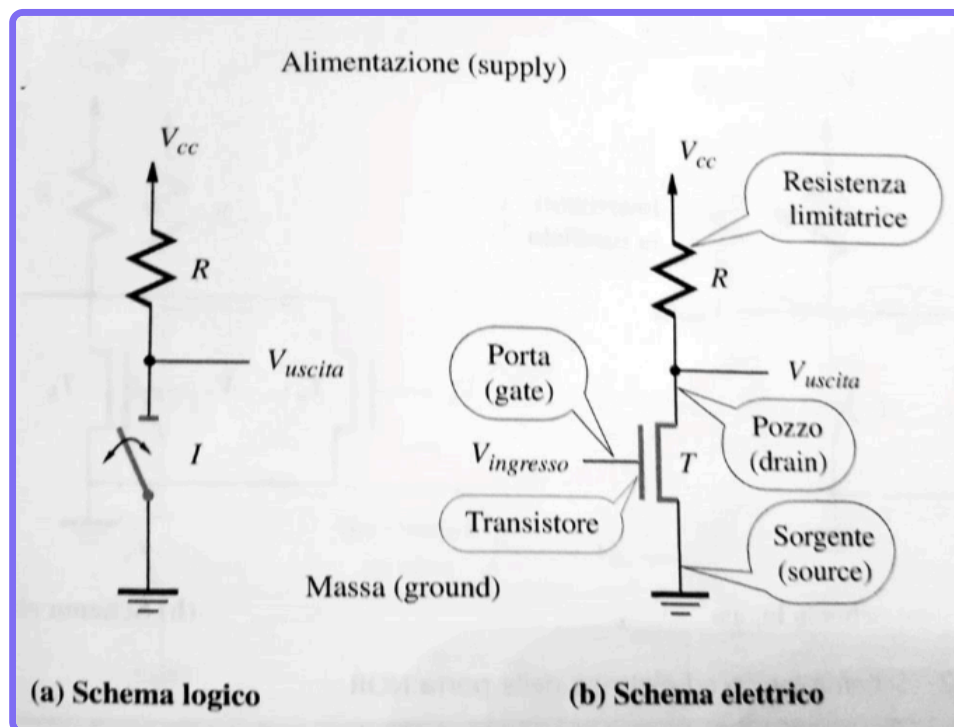
Tipi di transistor MOS

- **NMOS:**
 - Conduzione con tensione alta in ingresso.
 - Sorgente collegata a massa.
- **PMOS:**
 - Conduzione con tensione bassa in ingresso.
 - Sorgente collegata all'alimentazione.

Realizzazione di porte logiche con transistor

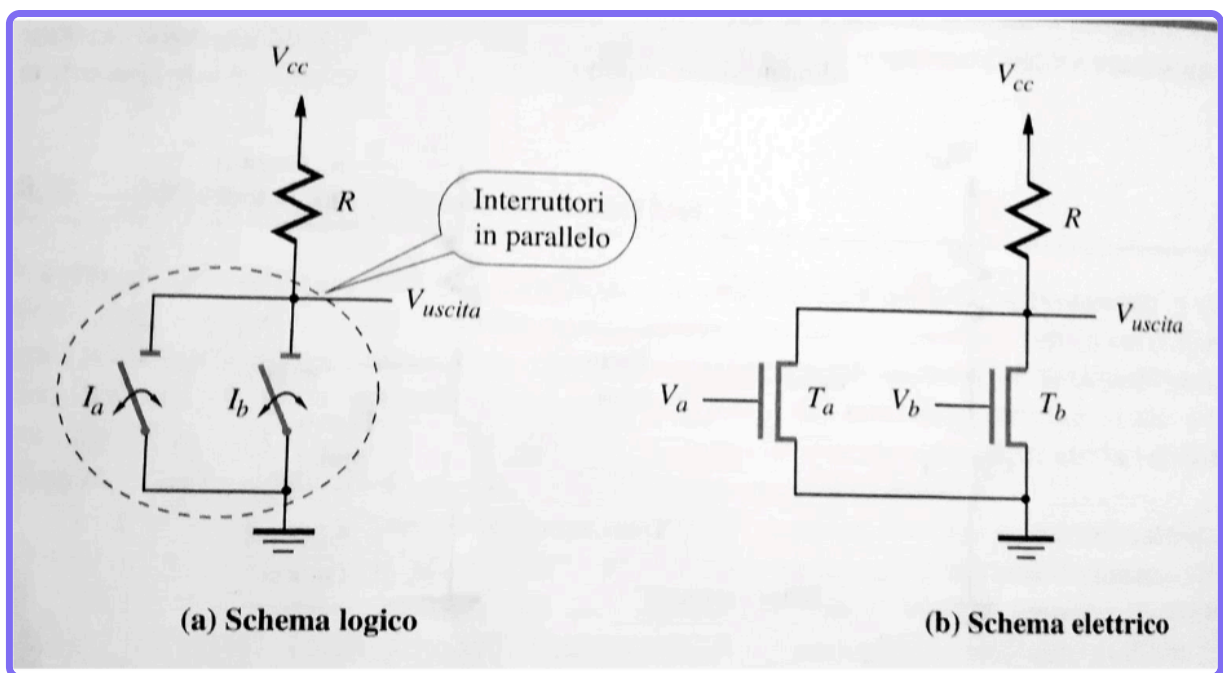
1. NOT:

- Un singolo NMOS inverte il segnale.
- Es.: ingresso = 1 → uscita = 0.



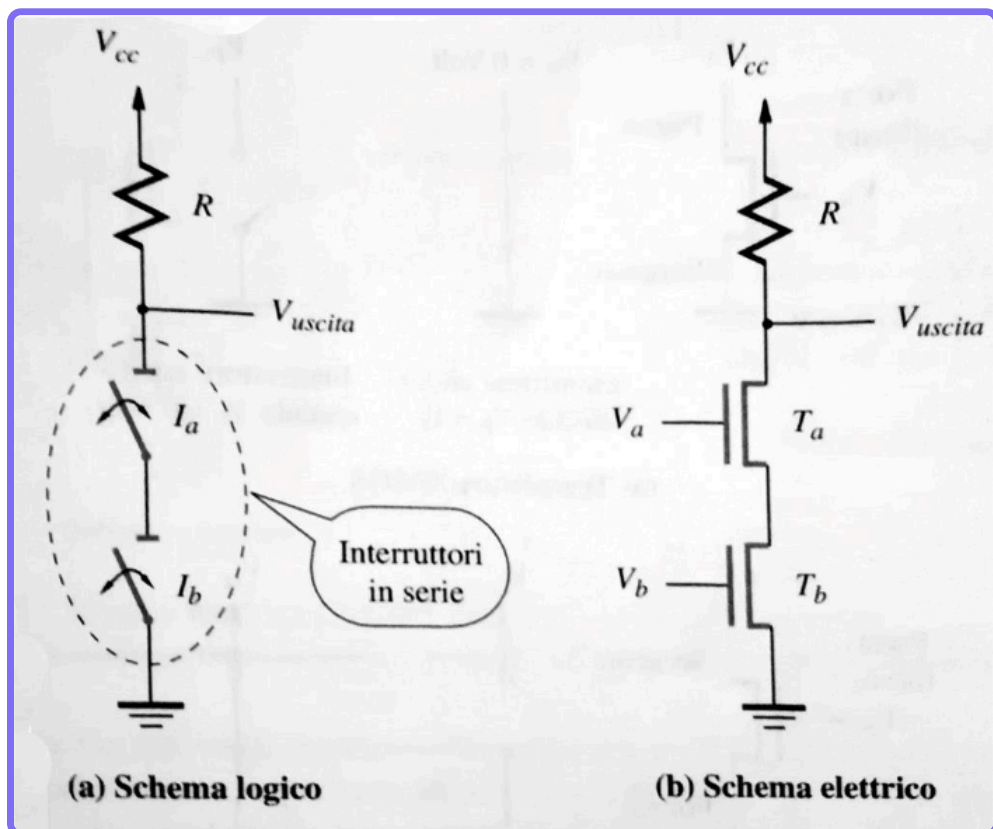
2. NOR:

- Due NMOS in parallelo.
- Es.: uscita = 1 solo se entrambi gli ingressi = 0.



3. NAND:

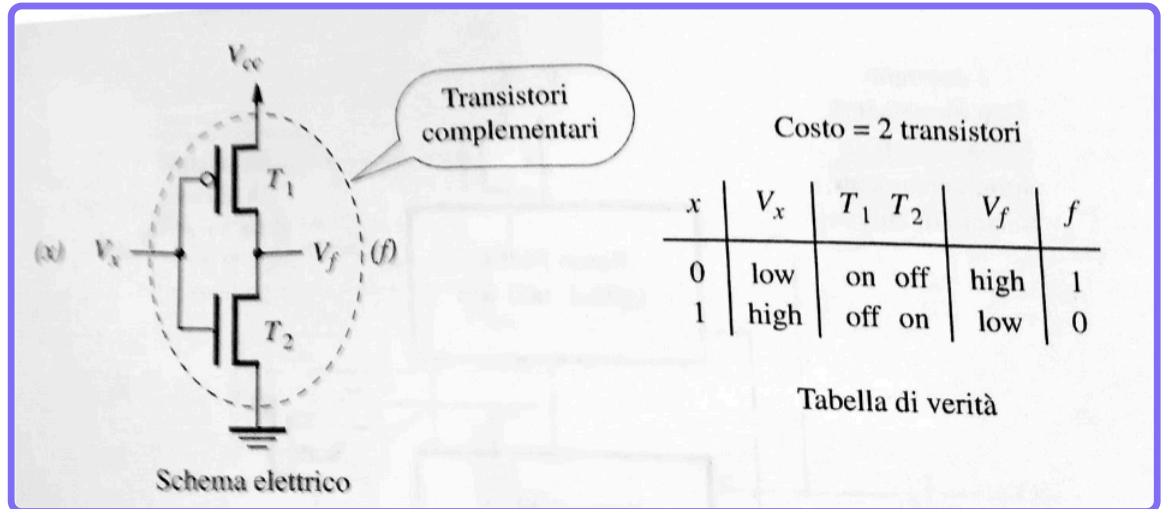
- Due NMOS in serie.
- Es.: uscita = 0 solo se entrambi gli ingressi = 1.



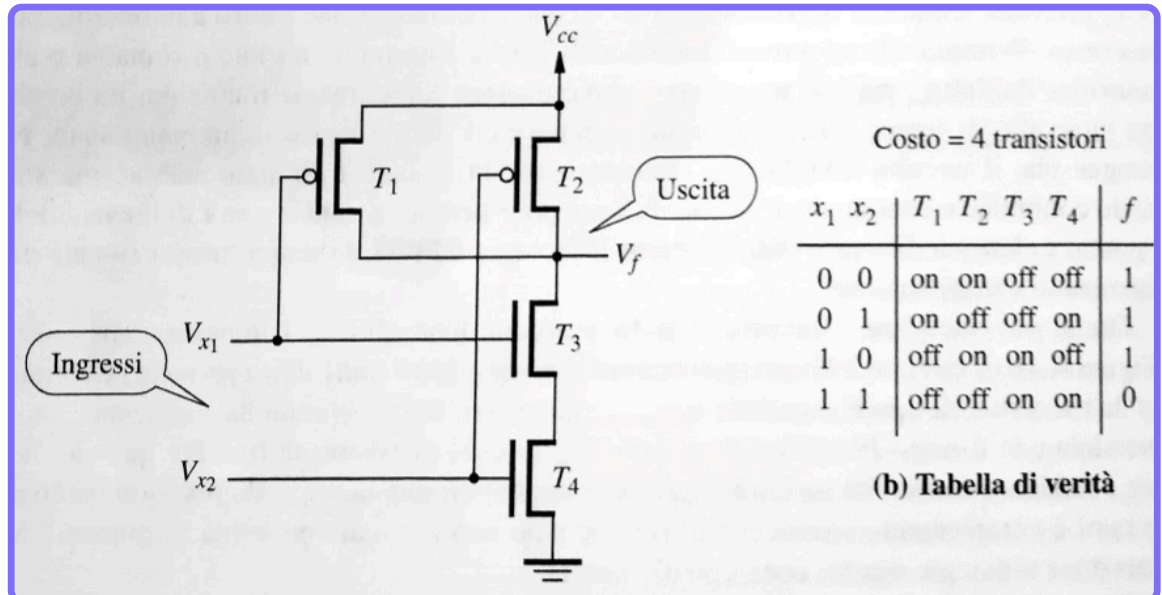
Tecnologia CMOS

- Combina NMOS e PMOS per migliorare efficienza e ridurre i consumi.
- Consumo di potenza solo durante la commutazione.
- Permette l'integrazione di miliardi di transistor su un chip.
- Offre alte frequenze di commutazione (nell'ordine dei GHz).

- Esempi:
 - NOT CMOS:**
 - Un NMOS e un PMOS in serie.

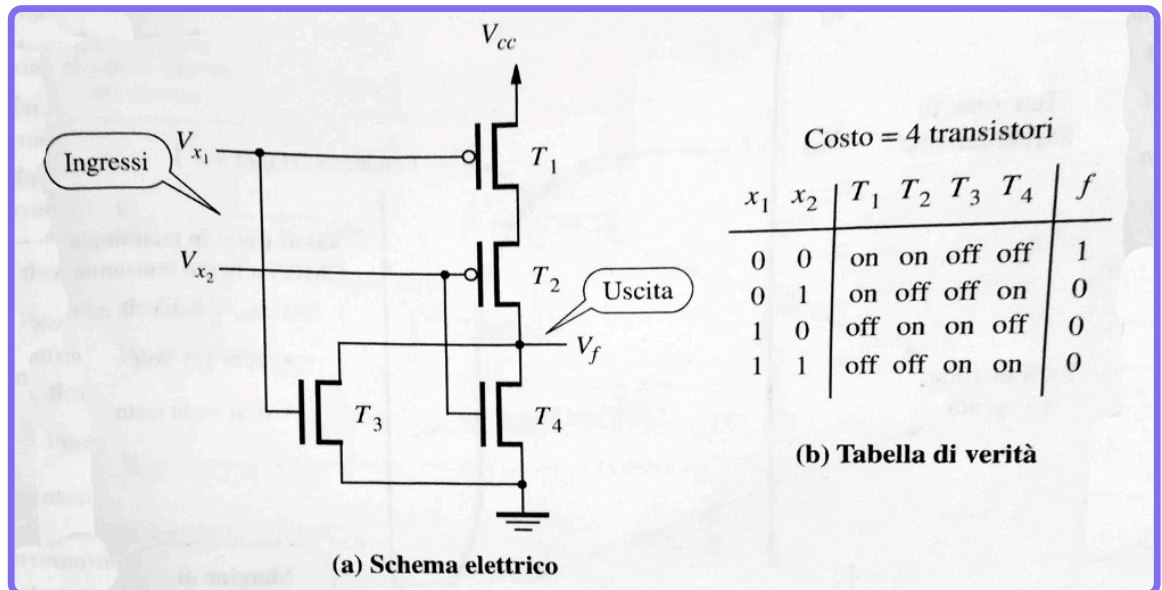


- NAND CMOS:**
 - Due NMOS in serie e due PMOS in parallelo.



- NOR CMOS:**

- Due NMOS in parallelo e due PMOS in serie.



Concetti avanzati nei circuiti logici

- **Ritardi nei circuiti:**
 - **Tempo di transizione:** Cambio di stato del segnale.
 - **Ritardo di propagazione:** Tempo necessario per aggiornare l'uscita.
 - **Percorso critico:** Determina la frequenza massima di lavoro.
- **Fan-in e Fan-out:**
 - **Fan-in:** Numero massimo di ingressi per una porta logica.
 - **Fan-out:** Numero massimo di uscite che una porta può pilotare.
 - Collegare più uscite a uno stesso ingresso può causare cortocircuiti o interferenze.

Circuiti integrati

- I circuiti integrati sono dispositivi in silicio che combinano componenti elettronici.
- Tipi di circuiti integrati:
 - **SSI:** Poche porte logiche.
 - **MSI:** Moduli funzionali (es. addizionatori).
 - **LSI:** Piccoli processori come la ALU.
 - **VLSI:** Processori potenti e memorie avanzate.
- Circuiti integrati possono essere:
 - **Analogici:** Elaborano segnali continui.
 - **Digitali:** Elaborano segnali discreti.
 - **Mixed:** Combinano analogico e digitale.

Porta tri-state

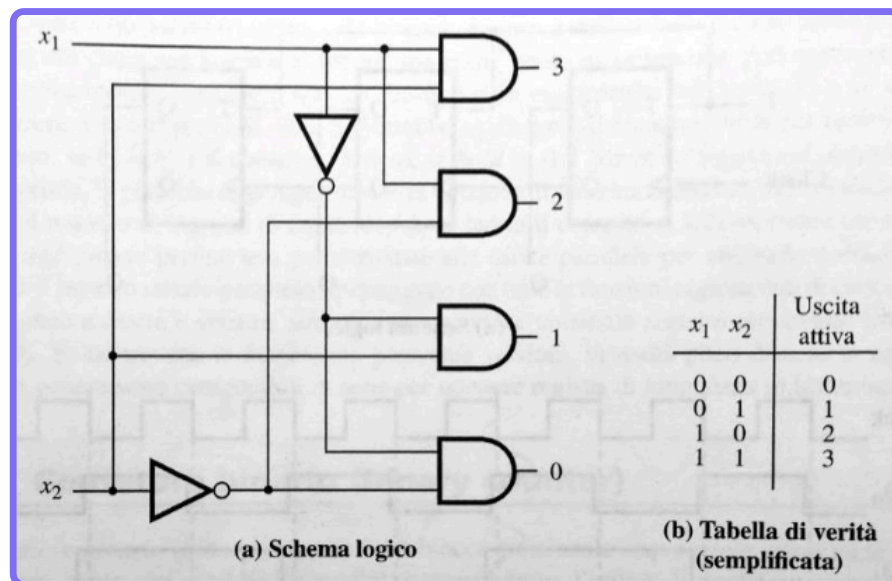
Questo tipo di porta può assumere 3 stati:

- Acceso
- Spento
- Alta impedenza

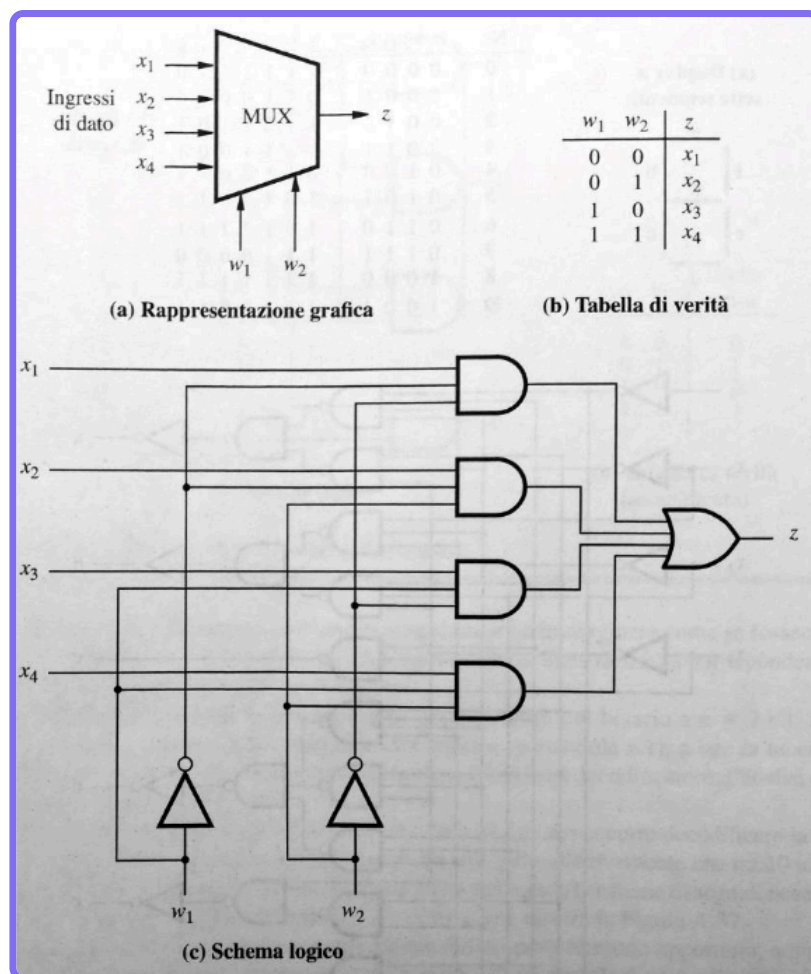
Quando la porta si trova in alta impedenza è come se fosse disconnessa, questa porta viene usata molto nei bus per permettere a più componenti di usare lo stesso bus ma senza influenzarsi tra di loro. [Spiegazione correlata al bus](#)

Decoder e Multiplexer

- **Decoder:** Converte un codice binario con n ingressi in 2^n uscite. In pratica prende in input un segnale e lo decodifica in segnali di uscita leggibili anche da altre componenti.



- **Multiplexer:** è un circuito digitale che seleziona uno tra i diversi ingressi e lo invia in uscita, basandosi sui segnali di controllo che gli arrivano, ciò permette a più fonti di condividere la stessa linea di connessione.



13_Laboratorio

1. Sistemi di numerazione

Per contare noi usiamo un sistema posizionale in base 10, invece i nostri dispositivi elettronici usano un sistema posizionale in base 2.

Conversione da decimale a binario

- Sia N il numero (in base 10) da convertire e in questo caso $B = 2$
- Si calcola la divisione intera $N = N/B$ e si mette da parte il resto R della divisione
- Se $N > 0$ si va al passo 2
- Se $N = 0$ si riportano i vari RESTI da destra verso sinistra: essi rappresentano il numero convertito in base B.

Operazione	Quoziente	Resto
13		
/2	6	1
/2	3	0
/2	1	1
/2	0	1

Lo stesso sistema può essere utilizzato per convertire un numero in base ottale (B=8) e anche esadecimale (B=16).

Numeri binari

I concetti on/off alla base dei nostri dispositivi possono essere rappresentati tramite i numeri binari:

- OFF = 0
- ON = 1

in generale un numero viene rappresentato tramite una stringa di bit, questi numeri binari possono essere rappresentati tramite 3 tecniche principali:

1. **Segno e valore assoluto:** si commuta il bit di segno da 0 a 1, mentre gli altri bit restano invariati.
2. **Complemento a uno:** si commuta qualsiasi bit da 0 a 1 e da 1 a 0
3. **Complemento a due:** si aggiunge 1 al complemento a uno

Queste 3 tecniche hanno in comune che il bit più a sinistra rappresenta il segno del numero:

- 0 se il numero è positivo
- 1 se il numero è negativo

Addizione e sottrazione di numeri in complemento a 2

Addizione: si applica il solito algoritmo di addizione

Sottrazione: si fa il complemento a 2 dei numeri che hanno il meno davanti e poi si calcola l'addizione algebrica

1	1	1	1		Catena di propagazione del riporto
	0	1	1	1	+ Primo addendo = +7 in comp. a 2
	1	1	0	1	= Secondo addendo = -3 in comp. a 2
1	0	1	0	0	Somma completa a 5 bit
↑					
	Somma a 4 bit = +4 in comp. a 2				
	Riporto in uscita (da trascurare)				

Estensione e riduzione

Accade spesso che i numeri vadano ridotti o estesi per adattarli a delle locazioni di memoria specifiche, per fare ciò seguiamo le seguenti regole:

- **Estendere:**
 - Se $n > 0$ aggiungiamo altri 0 a sinistra
 - Se $n < 0$ aggiungiamo altri 1 bit a sinistra
- **Ridurre:**
 - Se $n > 0$ togliere bit 0 da sinistra (smettere prima che emerge 1)
 - Se $n < 0$ togliere bit 1 da sinistra (smettere prima che emerge 1)

ESTENSIONE	RIDUZIONE
011 (3) → 0.011	00011 → 011
101 (-3) → 1..101	11101 → 101

Overflow

Il risultato di addizione e sottrazione in complemento a due è corretto se è COMPRESO nell'intervallo $[-2^{n-1}, 2^{n-1}]$ in caso contrario si ha Overflow. Il trabocco si verifica se e solo se i due addendi hanno lo stesso segno, ma il bit di segno della somma risulta diverso dal bit di segno degli addendi. Esempio: +7+4 con $n = 4$ si ottiene 1011 (-5)

Numeri in virgola mobile

Un numero in virgola mobile può essere rappresentato usando:

- Un **segno** s
 - La **mantissa** m
 - Un **esponente** e con segno in base 2
- In base allo standard utilizzato abbiamo un numero di bit usati:

- Standard IEEE 754 numeri 32 bit:

- ✓ 1 bit (segno)
- ✓ 8 bit (esponente)
- ✓ 23 bit (mantissa)

- Standard IEEE 754 numeri 64 bit

- ✓ 1 bit (segno)
- ✓ 11 bit (esponente)
- ✓ 52 bit (mantissa)

per rappresentare i numeri in questo modo faccio così:

Per rappresentare $13,25$ in questo modo procedo così

1) Convertito in binario

2) Sposto la virgola facendo diventare il numero del tipo $1,11 \cdot 2^E$

3) Aggiungo il bit all'esponente

1) $13,25 = 1101,01$

2) $1101,01 = 1,10101$ → scivolo nella mantissa

3) $3 + \frac{2^8}{2} - 1 = 130$ → 10000010 → scivolo nell'esponente

0 10000010 101010000000000000000000
 ↓ ↓ ↓
 segno esponente mantissa

Codice ASCII

Per rappresentare le lettere, i numeri e i simboli in generale viene utilizzato un alfabeto definito su 7 bit chiamato alfabeto ascii

Byte	Cod.	Char	Byte	Cod.	Char	Byte	Cod.	Char	Byte	Cod.	Char
00000000	0	Null	00100000	32	Spc	01000000	64	@	01100000	96	`
00000001	1	Start of heading	00100001	33	!	01000001	65	A	01100001	97	a
00000010	2	Start of text	00100010	34	"	01000010	66	B	01100010	98	b
00000011	3	End of text	00100011	35	#	01000011	67	C	01100011	99	c
00000100	4	End of transmit	00100100	36	\$	01000100	68	D	01100100	100	d
00000101	5	Enquiry	00100101	37	%	01000101	69	E	01100101	101	e
00000110	6	Acknowledge	00100110	38	&	01000110	70	F	01100110	102	f
00000111	7	Audible bell	00100111	39	'	01000111	71	G	01100111	103	g
00001000	8	Backspace	00101000	40	(	01001000	72	H	01101000	104	h
00001001	9	Horizontal tab	00101001	41	)	01001001	73	I	01101001	105	i
00001010	10	Line feed	00101010	42	*	01001010	74	J	01101010	106	j
00001011	11	Vertical tab	00101011	43	+	01001011	75	K	01101011	107	k
00001100	12	Form Feed	00101100	44	,	01001100	76	L	01101100	108	l
00001101	13	Carriage return	00101101	45	-	01001101	77	M	01101101	109	m
00001110	14	Shift out	00101110	46	.	01001110	78	N	01101110	110	n
00001111	15	Shift in	00101111	47	/	01001111	79	O	01101111	111	o
00010000	16	Data link escape	00110000	48	0	01010000	80	P	01110000	112	p
00010001	17	Device control 1	00110001	49	1	01010001	81	Q	01110001	113	q
00010010	18	Device control 2	00110010	50	2	01010010	82	R	01110010	114	r
00010011	19	Device control 3	00110011	51	3	01010011	83	S	01110011	115	s
00010100	20	Device control 4	00110100	52	4	01010100	84	T	01110100	116	t
00010101	21	Neg. acknowledge	00110101	53	5	01010101	85	U	01110101	117	u
00010110	22	Synchronous idle	00110110	54	6	01010110	86	V	01110110	118	v
00010111	23	End trans. block	00110111	55	7	01010111	87	W	01110111	119	w
00011000	24	Cancel	00111000	56	8	01011000	88	X	01111000	120	x
00011001	25	End of medium	00111001	57	9	01011001	89	Y	01111001	121	y
00011010	26	Substitution	00111010	58	:	01011010	90	Z	01111010	122	z
00011011	27	Escape	00111011	59	;	01011011	91	[	01111011	123	{
00011100	28	File separator	00111100	60	<	01011100	92	\	01111100	124	
00011101	29	Group separator	00111101	61	=	01011101	93	]	01111101	125	}
00011110	30	Record Separator	00111110	62	>	01011110	94	^	01111110	126	~
00011111	31	Unit separator	00111111	63	?	01011111	95	_	01111111	127	Del

2. Istruzioni macchina

Istruzione	Sintassi	Descrizione
Load	R2, LOC	Legge i contenuti di una locazione di memoria il cui indirizzo è rappresentato simbolicamente dall'etichetta LOC e carica i contenuti nel registro R2. I contenuti originali della locazione LOC sono preservati mentre quelli del registro R2 sono sovrascritti.
Add	R4, R2, R3	Somma i contenuti dei registri R2 e R3 e inserisce la loro somma nel registro R4. Gli operandi in R2 e R3 non sono alterati, ma il precedente valore in R4 è sovrascritto dalla somma.
Store	R4, LOC	Copia l'operando del registro R4 nella locazione di memoria LOC. I contenuti originali della locazione LOC sono sovrascritti mentre quelli del registro R4 sono preservati.
Move	R4, R3	Carica il contenuto del registro R3 in R4.
Subtract	R4, R2, R3	Sottrae i contenuti dei registri R2 e R3 e inserisce il risultato nel registro R4.

Modi di indirizzamento

- **Modo di registro:** L'operando o il risultato è contenuto in un registro di processore, il cui nome (che è indirizzo) è dato nell'istruzione. ES: ADD R4,R2,R3

- **Modo assoluto:** L'operando o il risultato è contenuto in una parola di memoria il cui indirizzo è dato nell'istruzione. ES: `LOAD R2, NUM1`
- **Modo immediato:** L'operando è dato esplicitamente nell'istruzione. ES: `R4, R6, #200`
- **Modo indiretto:** Il nome di un registro di processore contenente l'INDIRIZZO di memoria dell'operando o del risultato è dato nell'istruzione. ES: `Load R2, (R5)` Il registro R5 agisce come puntatore
- **Modo con indice e spiazzamento:** L'indirizzo effettivo di operando o risultato è ottenuto addizionando un valore costante (spiazzamento) al contenuto di un registro (indirizzo). ES: `LOAD R2, 20(R5)`

Istruzioni da usare su visual

Istruzione	Descrizione
<code>LDR Ri, [Rj]</code>	Carica nel registro Ri il contenuto della parola di memoria puntata da Rj.
<code>ADD Rd, Ri, Rj</code>	Somma il contenuto di Ri e Rj e lo carica nel registro Rd.
<code>SUB Rd, Ri, Rj</code>	Sottrae il contenuto di Rj da Ri e lo carica nel registro Rd.
<code>STR Ri, [Rj]</code>	Salva nella parola di memoria puntata da Rj il contenuto di Ri.
<code>MOV Ri, Rj / MOV Ri, #Valore</code>	Carica nel registro Ri il contenuto di Rj o il valore immediato.

Direttive di assembler

Le direttive di assembler servono per dare informazioni utili all'assembler.

- **EQU:** associa il valore numerico nome (in visual: EQU)
- **ORIGIN:** indica l'indirizzo di partenza dove inserire le istruzioni
- **RESERVE:** indica all'assembler di riservare uno spazio di memoria espresso in byte (in visual: FILL)
- **DATAWORD:** Inizializza una word di memoria con il contenuto passato (in visual: DCD)
- **END:** indica all'assembler la fine del testo del programma sorgente (in visual: END)

Bit di esito

Il processore tiene traccia di alcune informazioni ausiliarie relative all'esito di svariate operazioni, che servono come condizioni per l'istruzione di salto. Tale informazioni sono espressi sotto forma di insiemi di bit detti bit di esito.

Bit di esito	Significato
N (negativo)	1 se risultato negativo, 0 se positivo o nullo
Z (zero)	1 se risultato nullo, 0 altrimenti
C (riporto)	1 se trabocco in binario naturale, 0 altrimenti (Carry)
V (trabocco)	1 se trabocco in comp. a due, 0 altrimenti (oVerflow)

Le istruzioni sottostanti aggiornano i bit di stato

Istruzione	Descrizione
ADDS Rd, Ri, Rj	Somma il contenuto di Ri e Rj e lo carica nel registro Rd (aggiorna i bit di stato NZCV).
SUBS Rd, Ri, Rj	Sottrae il contenuto di Rj da Ri e lo carica nel registro Rd (aggiorna i bit di stato NZCV).
CMP Ri, Rj	Sottrae Rj da Ri e aggiorna i bit di stato. Il risultato viene scartato.
CMN Ri, Rj	Somma Ri e Rj e aggiorna i bit di stato. Il risultato viene scartato.

Istruzione di salto condizionato

Branch_if_ condizione destinazione_salto

nel caso la **condizione** sia vera il programma salta alla **destinazione_salto** espressa come locazione di memoria contenente l'istruzione da eseguire nel caso la condizione sia vera.

Codice	Significato
EQ (equal)	Z
NE (not equal)	$\neg Z$
GE (greater-equal)	$\neg(N \oplus V)$
LT (lower than)	$N \oplus V$
GT (greater than)	$\neg(Z \text{ OR } (N \oplus V))$
LE (lower equal)	$Z \text{ OR } (N \oplus V)$

Le lettere usate sono le lettere dei corrispondenti bit di esito.

3. Algebra Booleana

Operatori logici

L'algebra booleana è un sistema algebrico in cui ogni variabile può assumere solo 2 valori (0 e 1) le principali operazioni definite sono:

- **OR**: che si denota con "+" o "V" ed è una funzione che vale 1 se almeno uno dei suoi ingressi è 1
- **AND**: che si denota con "*" o "^" ed è una funzione che vale 1 se entrambi i suoi ingressi valgono 1

- **NOT:** che si denota con "¬" o "¬" ed è una funzione che inverte il valore del suo ingresso
- **XOR:** che si denota con " $\oplus$ " è una funzione che vale 1 solo se il numero di uno nei suoi ingressi è dispari

A	B	A OR B	A AND B	NOT A	A XOR B
0	0	0	0	1	0
0	1	1	0	1	1
1	0	1	0	0	1
1	1	1	1	0	0

Abbiamo anche altre proprietà importanti che riguardano questi operatori logici:

REGOLA	FORMA DUALE
Proprietà Distributiva	
$x + y \cdot z = (x + y) \cdot (x + z)$	$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$
Proprietà Di idempotenza	
$x + x = x$	$x \cdot x = x$
Proprietà di complemento	
$x + \neg x = 1$	$x \cdot \neg x = 0$
Proprietà dello 1 e dello 0	
$1 + x = 1$	$0 \cdot x = 0$

Ricordiamo anche il teorema di De Morgan che ci dice:

- $\neg(x + y) = \neg x \cdot \neg y$
- $\neg(x \cdot y) = \neg x + \neg y$

Prima forma canonica (somma di prodotti)

Si crea la tabella di verità di un'espressione logica e poi si seguono i seguenti step:

1. Si individuano i casi in cui il risultato è pari a **1**
2. Per ogni caso si costruisce un prodotto delle n variabili, ogni variabile è
 - se uguale a 1 la scriviamo direttamente
 - se uguale a 0 la scriviamo negata
3. Si sommano tra loro i prodotti ottenuti

x_1	x_2	x_3	f_1
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

$f_1 = \overline{x_1} \overline{x_2} \overline{x_3} + \overline{x_1} \overline{x_2} x_3 + \overline{x_1} x_2 x_3 + x_1 x_2 x_3$

Seconda forma canonica (prodotto di somme)

Si crea la tabella di verità di un'espressione logica e poi si seguono i seguenti step:

1. Si individuano tutti i casi in cui il risultato è pari a **0**
2. Per ogni caso, si costruisce una somma delle n variabili, ogni variabile è:
 - se uguale a 0 la scriviamo direttamente
 - se uguale a 1 la scriviamo negata
3. Moltiplichiamo le somme ottenute.

x_1	x_2	x_3	f_1
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

$f_1 = (x_1 + \bar{x}_2 + x_3) \cdot (\bar{x}_1 + x_2 + x_3) \cdot (\bar{x}_1 + x_2 + \bar{x}_3) \cdot (\bar{x}_1 + \bar{x}_2 + x_3)$

Forma minima

Due espressioni logiche si dicono equivalenti se hanno la medesima tabella di verità, un'espressione si dice in **forma minima** quando non esiste un'altra espressione equivalente ma di costo inferiore, il costo di un'espressione si calcola contando il numero di volte in cui le variabili compaiono, per noi è molto importante ridurre un'espressione perché un'espressione minima produrrà un circuito logico più semplice.

≡ Esempio

$$(x_1 + x_2) \cdot (x_1 + x_2) \cdot (x_1 + x_2) \rightarrow \text{costo } 6$$

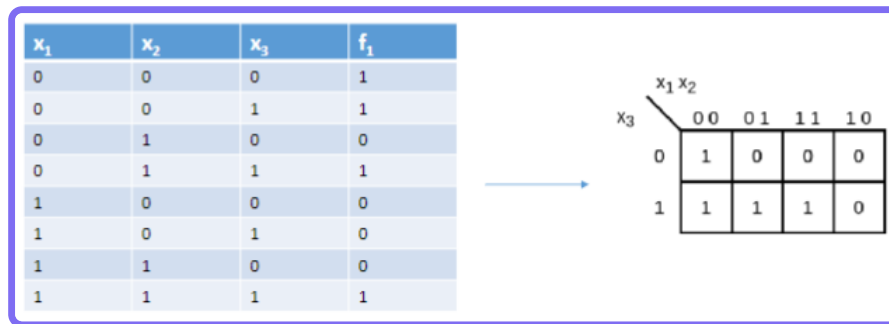
$$x_1 \cdot x_2 \rightarrow \text{costo } 2$$

Di seguito un'esempio di minimizzazione

$$\begin{aligned} \bar{x}_1\bar{x}_2\bar{x}_3 + \bar{x}_1\bar{x}_2x_3 + \bar{x}_1x_2\bar{x}_3 + x_1x_2\bar{x}_3 &= \\ = \bar{x}_1\bar{x}_2(\bar{x}_3 + x_3) + x_2\bar{x}_3(\bar{x}_1 + x_1) &= \text{(distributiva)} \\ = \bar{x}_1\bar{x}_2 \cdot 1 + x_2\bar{x}_3 \cdot 1 &= \text{(complemento)} \\ = \bar{x}_1\bar{x}_2 + x_2\bar{x}_3 &= \text{(forma minima)} \end{aligned}$$

Mappe di Karnaugh

Le mappe di Karnaugh sono un metodo di tipo geometrico che permettono di ricavare l'espressione logica di costo minimo data una funzione



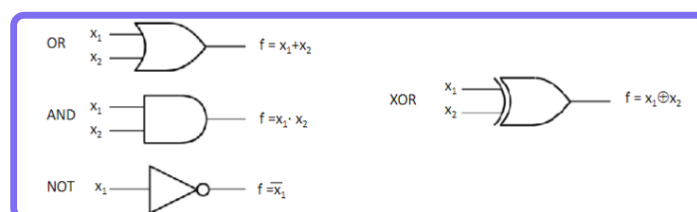
Come si usano:

1. Trasformare la tavola di verità in una mappa di Karnaugh
2. Raggruppare le caselle di valore 1 adiacenti orizzontalmente o verticalmente
3. Ogni gruppo rappresenta il prodotto delle sue variabili con lo stesso valore (forma diretta se 1 e negata se 0). Identificare quali variabili non contribuiscono ed eliminarle nella forma analitica.
4. Somma dei gruppi creati

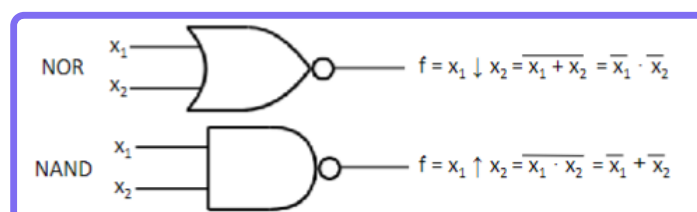
Condizione di indifferenza: spesso accade che una funzione logica non sia definita su tutte le combinazioni di valori delle sue variabili, nella tabella di verità sono indicate con il simbolo "X" e quindi anche nella mappa di karnaugh corrispondente avranno un x, il loro valore va scelto in modo arbitrario cercando di creare una semplificazione a costo minimo.

Circuiti logici

Le operazioni logiche possono essere realizzate da semplici circuiti elettronici chiamati porte. Collegando varie porte tra di loro si possono rappresentare le espressioni logiche. Questi sono i simboli principali.



Altre porte molto importanti sono le porte NAND e NOR, si denota tramite gli operatori a due argomenti " $\uparrow$ " o " $\downarrow$ " rispettivamente



A	B	$A \uparrow B$	$A \downarrow B$
0	0	1	1
0	1	0	1
1	0	0	1
1	1	0	0

Queste 2 porte godono della proprietà commutativa ma non di quella associativa e appunto per questo non si possono creare degli alberi di porte. Grazie alle leggi di De Morgan e alla legge di involuzione possiamo passare da una SOP ad una rete di NAND

4. La pila

La pila consiste in una lista di elementi alla quale però si può lavorare solo agendo sulla cima della pila. È possibile eseguire solo 2 operazioni:

- **PUSH:** aggiungiamo un elemento in cima

Subtract SP, SP,#4 ;diminuire l'indirizzo contenuto in SP di una parola per puntare alla nuova cima

Store Rj, (SP) ; scrivere il valore richiesto nella parola puntata da SP

- **POP:** togliere un elemento dalla cima

Load Rj, (SP) ;copiare il valore contenuto nella locazione di memoria puntata da SP in un registro del processore

Add SP, SP,#4 ;aumentare l'indirizzo contenuto in SP di una parola per puntare alla nuova cima

La pila in Visual

Istruzione	Operazione	Note
STM[codice] SP[!] {lista di registri}	PUSH	Store Multiple Registro di ordine più basso in cima
LDM [codice] SP[!] {lista di registri}	POP	Load Multiple Registro di ordine più basso dalla cima

Questi 2 comandi vanno a salvare nella pila la lista dei registri che inseriamo, l'ordine con la quale vengono inseriti si basa sul codice passato, i codici disponibili sono:

Codice	Descrizione
FD	Full descending
FA	Full ascending
ED	Empty descending
EA	Empty ascending
!	Aggiorna il valore del puntatore alla cima a fine operazione

≡ Esempio

I valori nei nostri registri sono:

Questo succede se uso STM con i vari codici:

FD {R0,R1} FD {R0,R1} FA {R0,R1} ED {R0,R1} EA {R0,R1}

FD {R2,R3} FD {R3,R2} FA {R2,R3} ED {R2,R3} EA {R2,R3}

FD {R4,R5} FD {R4,R5} FA {R4,R5} ED {R4,R5} EA {R4,R5}

SP →	26
	28
SP →	22
	24
SP →	18
	20

SP →	26
	28
SP →	22
	24
SP →	18
	20

	18
SP →	20
	22
SP →	24
	26
SP →	28

SP →	
	26
SP →	28
	22
SP →	24
	18
	20

	18
	20
SP →	22
	24
SP →	26
	28
SP →	

Questo succede se uso LDM:

Initial State

SP →	26	R0	0
	28	R1	0
	22	R2	0
	24	R3	0
	18	R4	0
	20	R5	0

LDM {R0,R1} = LDM {R1,R0}

	26	R0	26
	28	R1	28
SP →	22	R2	0
	24	R3	0
	18	R4	0
	20	R5	0

LDM {R3,R4}

	26	R0	26
	28	R1	28
	22	R2	0
	24	R3	22
SP →	18	R4	24
	20	R5	0

Area di attivazione in pila

Il blocco di programma riservato ai sottoprogrammi è anche essa una pila ed è chiamato **Stack Frame**, il **Frame Pointer** è un registro che punta allo stack frame del sottoprogramma in esecuzione.

Gestione dei sottoprogrammi

Un sottoprogramma è una lista di istruzioni che eseguono un compito specifico e che possono essere richiamate in un qualsiasi punto del programma.

- **Chiamata a sottoprogramma:** viene fatta attraverso l'istruzione `call INDIRIZZO`, l'operazione di chiamata esegue 2 step:
 1. Salva il contenuto del registro PC nel Link Register;
 2. Salta all'indirizzo di destinazione indicato nell'istruzione di chiamata

- **Rientro da sottoprogramma:** viene fatta attraverso l'istruzione `RETURN`, quello che fa questa istruzione è salvare il contenuto del link register nel registro PC

In Visual facciamo così:

BL	Indirizzo sottoprogramma	Salva il PC nel LR prima di effettuare il salto	Tre registri speciali: R13 = SP R14 = LR R15 = PC
MOV	PC, LR	Rientra da sottoprogramma	

Il passaggio di parametri ai sottoprogrammi può essere fatto attraverso i registri o usando la pila.

Programma chiamante			
	Load	R2, N	Il parametro 1 è la dimensione della lista
	Move	R4, #NUM1	Il parametro 2 è la locazione della lista
	Call	TOTALE	Chiama il sottoprogramma
	Store	R3, SOMMA	Immagazzina il risultato
Sottoprogramma			
TOTALE:	Subtract	SP, SP, #4	Salva in pila R5
	Store	R5, (SP)	
	Clear	R3	Inizializza la somma a 0
CICLO:	Load	R5, (R4)	Preleva il prossimo numero
	Add	R3, R3, R5	Aggiungi questo numero alla somma
	Add	R4, R4, #4	Incrementa di 4 il puntatore
	Subtract	R2, R2, #1	Decrementa il contatore
	Branch_if_[R2]>0	CICLO	
	Load	R5, (SP)	Ripristina il contenuto di R5
	Add	SP, SP, #4	
	Return		Rientra al programma chiamante

5. Shifting e comandi vari

And	R4,R2,R3	Calcola l' AND bit a bit degli operandi nei registri R2 e R3 e lascia il risultato in R4	AND
Or	R4,R2,R3	Calcola l' OR bit a bit degli operandi nei registri R2 e R3 e lascia il risultato in R4	ORR
XOr	R4,R2,R3	Calcola l' XOR bit a bit degli operandi nei registri R2 e R3 e lascia il risultato in R4	EOR
LShiftL	Ri, Rj, contatore	Shift logico a sinistra. Scorre il contenuto di Rj a sinistra di un numero di posizioni di bit dato dall'operando contatore e pone il risultato in Ri. Gli spazi vuoti si riempiono con 0	LSL
LShiftR	Ri, Rj, contatore	Shift logico a destra. Scorre il contenuto di Rj a destra di un numero di posizioni di bit dato dall'operando contatore e pone il risultato in Ri. Gli spazi vuoti si riempiono con 0	LSR

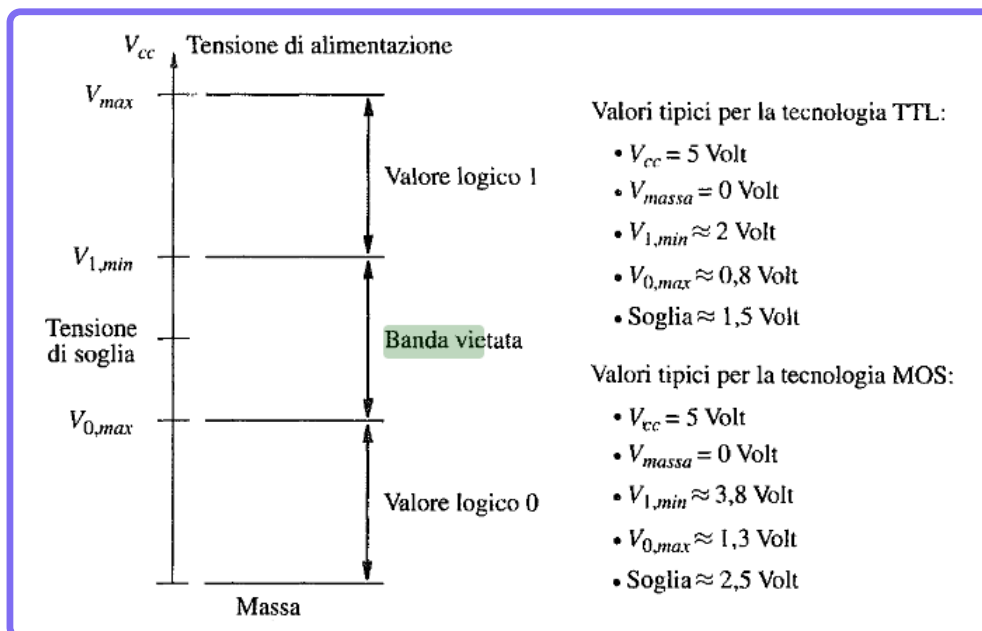
AShiftR	R4,R2,#2	Shift aritmetico a destra. Scorre il contenuto di Rj a sinistra di un numero di posizioni di bit dato dall'operando contatore e pone il risultato in Ri. Riempie le posizioni lasciate libere con il valore del bit più significativo	ASR
AShiftL	R4,R2,#2	Shift aritmetico a sinistra. Identico allo scorrimento logico verso sinistra	LSL
RotateL	R4,R2,#2	Rotazione verso sinistra	-
RotateR	R4,R2,#2	Rotazione verso destra	ROR

LoadByte	Rdst, LOCBYTE	Legge un singolo byte dalla memoria e lo registra negli 8 bit meno significativi del registro destinazione mettendo a 0 gli altri bit	LDRB
StoreByte	Rsrc, LOCBYTE	Salva gli 8 bit meno significativi del registro sorgente nella locazione di memoria specificata	STRB
Multiply	Rk,Ri,Rj	Effettua la moltiplicazione tra due numeri in complemento a due contenuti nei registri Ri e Rj e salva il risultato in Rk. In genere vengono salvati in Rk i bit meno significativi del prodotto e i più significativi non vengono calcolati	-
Divide	Rk,Ri,Rj	Effettua la divisione intera tra due numeri in complemento a due contenuti nei registri Ri e Rj e salva il risultato in Rk	-

6. Circuiti integrati

Transistor

Nei circuiti elettronici, per rappresentare i valori 0 e 1 delle variabili binarie si usano i valori di tensione elettrica, si stabilisce una **tensione di soglia** dove tutti i valori di tensione rappresentano un 1 mentre quelli inferiori rappresentano uno 0, i valori vicini alla soglia sono ambigui e quindi per evitare incertezze nasce la **banda vietata** dove i valori non vengono presi in considerazione.



I transistor sono delle componenti elettroniche che svolgono il compito di interruttori, in specifico i transistor MOS (transistore a metallo-ossido semiconduttore) sono quelli più utilizzati nei nostri dispositivi, questo tipo di transistor hanno 3 collegamenti: **Base**, **Pozzo** e **Sorgente**, a seconda della tensione in ingresso nella Base il transistore collegherà o meno la

sorgente al pozzo, se il transistor sta conducendo allora la tensione in entrata è uguale alla tensione in uscita, esistono 2 tipi di transistor MOS:

- **NMOS:**
 - il transistor è in conduzione (vale 1) se $V_b = V_{cc}$
 - il transistor è in interdizione (vale 0) se $V_b = 0$
 - **PMOS:**
 - il transistor è in conduzione (vale 1) se $V_b = 0$
 - il transistor è in interdizione (vale 0) se $V_b = V_{cc}$
- Attraverso questi 2 transistor possiamo costruire le seguenti porte logiche:
- **NOT:** si ottiene una porta NOT con un transistor NMOS collegando la sorgente alla massa e il pozzo all'alimentazione tramite una resistenza, per una tensione di ingresso di "1" in uscita avremo una tensione di "0" e viceversa.
 - **NOR:** si ottiene collegando due transistor NMOS in parallelo, solo se entrambi i transistori sono in interdizione la tensione in uscita sarà uno altrimenti sarà sempre 0
 - **NAND:** collegando due transistor NMOS in serie si ottiene una porta NAND
 - **AND:** collegando in serie una NAND e NOT
 - **OR:** collegando in serie una NOR e NOT

L'unico problema dei transistor NMOS è che consumano tanta corrente, così nascono i transistor CMOS che hanno un consumo di potenza ridotto e soprattutto dimensioni ridotte infatti da quando sono nati abbiamo componenti con miliardi di transistor all'interno

Definizioni nelle porte logiche

- il **tempo di transizione** in un circuito è il tempo impiegato da un segnale per transitare di livello
- il **ritardo di propagazione** è il tempo che ci mette il nostro circuito ad adattarsi ai nuovi valori in input
- la **frequenza di lavoro** è il numero di volte che esso commuta in un determinato tempo.
- **Fan-in** è il numero di ingressi di una porta logica
- **Fan-out** è il numero di uscite di una porta logica

Tip

Tipicamente il fan-in e fan-out si limita a meno di 10 per porta perché un numero più elevato inciderebbe sul ritardo di propagazione

Circuiti integrati

Decodificatore: un blocco funzionale in grado di decodificare un codice binario in ingresso

Multiplatore: un blocco funzionale dotato di molteplici ingressi di selezione e altrettanti

ingressi dato e di una sola uscita, quello che fa questo blocco funzionale è selezionare uno degli ingressi dato basandosi sui segnali che arrivano negli ingressi selezione e inviarlo in uscita

Reti sequenziali

Le reti che riescono a memorizzare informazioni sono chiamate **reti sequenziali**, queste reti logiche sono basate sui cicli, una rete sequenziale in grado di memorizzare dei bit viene anche detta **bistabile** ne esistono di 3 tipi:

- **Bistabile asincrono:** Sono privi di un segnale di sincronismo e modificano il loro stato in seguito al cambiamento degli ingressi.
- **Bistabile sincrono:** Sono sensibili ad un segnale di controllo e i cambiamenti di stato avvengono solo in corrispondenza di un impulso del segnale di controllo
- **Bistabile di tipo D:** Memorizza un bit unificando gli ingressi S e R in un unico ingresso D. Quando il segnale di clock CLK è alto ($CLK = 1$), l'uscita segue il valore di D; quando è basso ($CLK = 0$), lo stato rimane invariato.